

The Exploration–Exploitation Dilemma

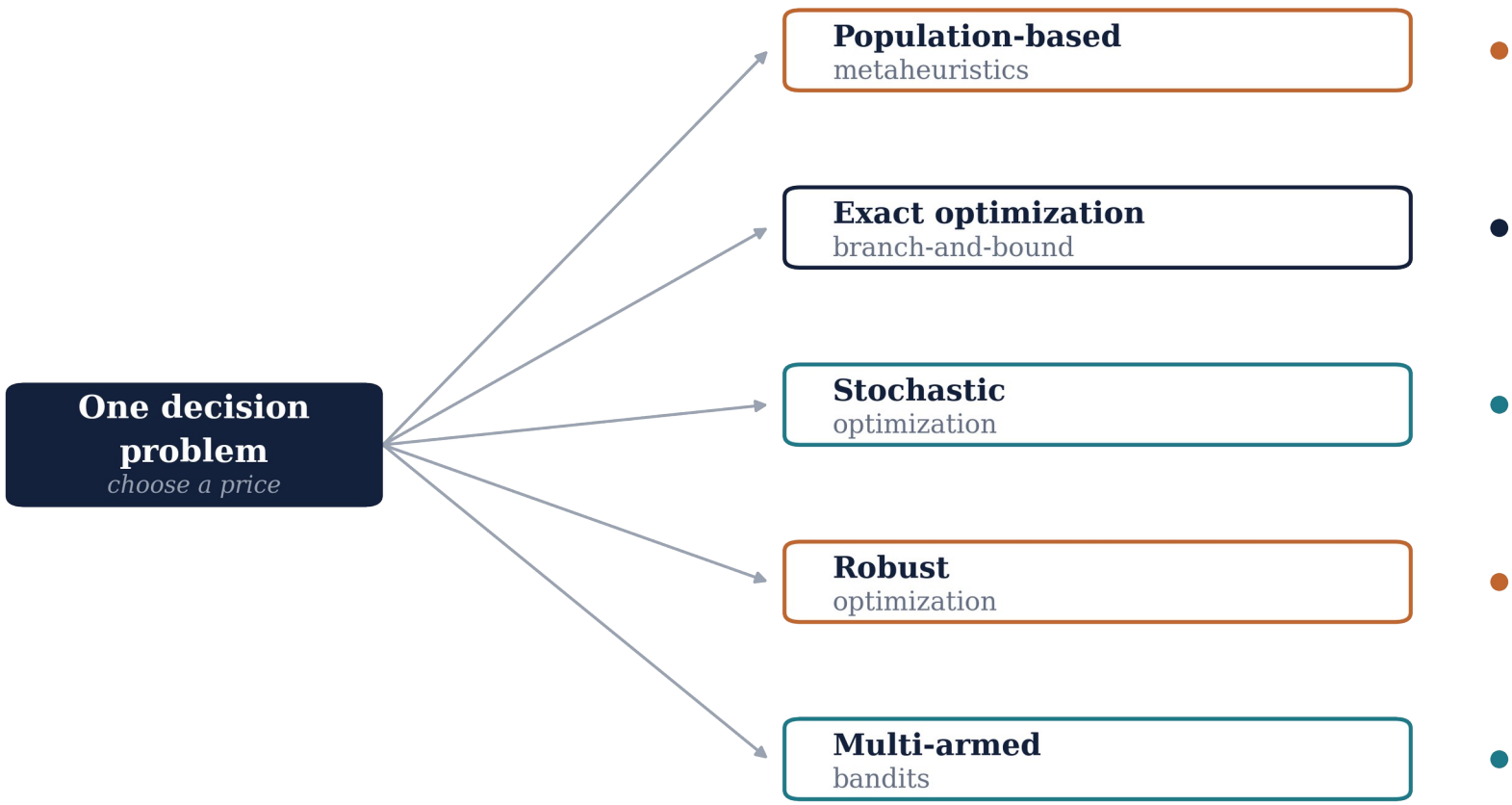
Artin (Alireza) Kavoosi

School of Industrial Engineering, College of Engineering

How should an intelligent decision-maker balance learning what it does not know with using what it already believes?

Every method in this lecture — population search, exact optimization, stochastic and robust programming, and multi-armed bandits — is a different answer to exactly this question.

One problem, five lenses



Each lens keeps the same underlying decision but changes what is assumed to be known, what is assumed to be random, and how aggressively the method is willing to commit before all the evidence is in.

HOW TO READ THIS DECK

The same twelve questions, every method

1 Understand

- Definition and intuition
- Mathematical formulation
- Step-by-step mechanism
- A worked example on the pricing problem

2 Diagnose

- Where exploration happens
- Where exploitation happens
- The source of the trade-off
- Key assumptions

3 Evaluate

- Strengths and weaknesses
- Typical failure modes
- Computational characteristics
- What it teaches us about explore vs. exploit

Example

Choose one price from {8, 10, 12, 14, 16} to maximize profit
The same five candidates will be seen through five different lenses.

Price	8	10	12	14	16
Profit proxy	4	5	7	6	4

WHY SUCH A SIMPLE PROBLEM

What changes from method to method is never the problem itself, only **what the method assumes it knows, what it treats as random, and how it decides when to stop looking and commit.**



PART 1 OF 8

Foundations

What exploration and exploitation actually mean, why the tension between them is unavoidable, and the vocabulary we will reuse across every method that follows.

Why this tension is unavoidable

THE DILEMMA

Any time a decision-maker has incomplete information and must act repeatedly or irreversibly, it faces the same choice: gather more information, or use what it already has.

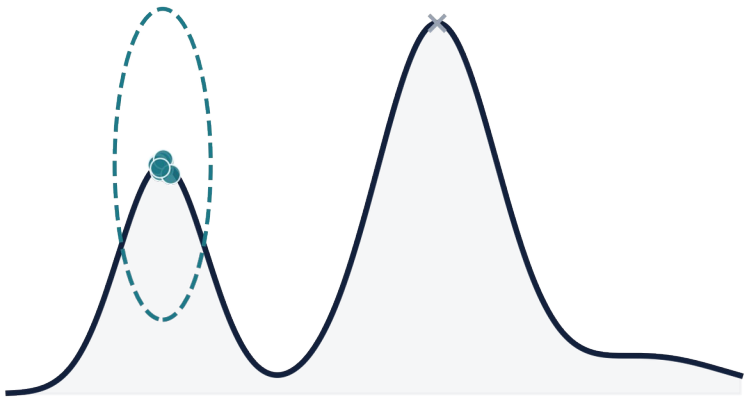
- **Foraging animals** deciding whether to revisit a known food patch or search for a better one.
- **Clinical trials** deciding whether to keep testing a promising drug or roll it out.
- **Reinforcement learning agents** deciding whether to try a new action or repeat their best one.
- **Firms** deciding whether to test a new price or keep charging the one that already works.

Two ways to fail

Every method below is, at its core, a different engineering answer to avoiding both failure modes at once.

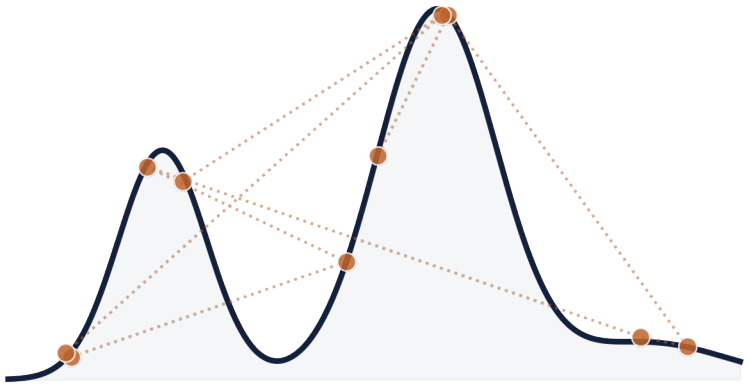
Too little exploration

population collapses here
global peak never visited



Too little exploitation

search keeps wandering
never commits to a peak



Too little exploration locks onto the first good option and never learns a better one exists.
Too little exploitation keeps sampling forever and never converts knowledge into results.

Core vocabulary

TWO DEFINITIONS

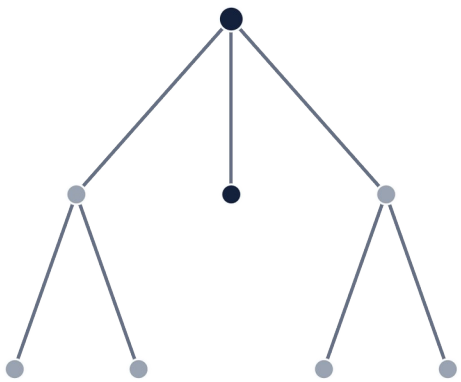
Exploration is the deliberate act of gathering information about alternatives whose value is still uncertain. Exploitation is the act of acting on the best information currently available.

- Every method in this lecture performs some version of both.
- The two trade off: resources spent exploring cannot simultaneously exploit, and vice versa.
- What differs across methods is **how** exploration is triggered, **what** is exploited, and **when** the balance shifts.
- This trade-off is not a design flaw — it is a mathematical consequence of acting under incomplete information.

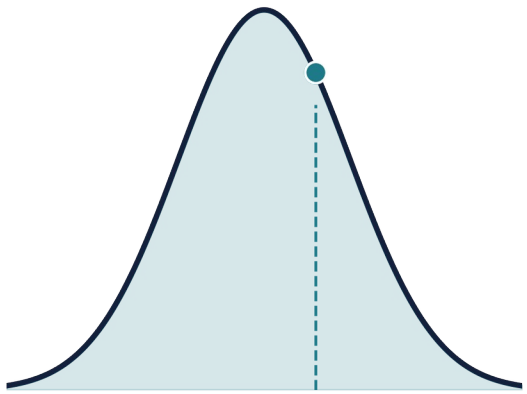
Where does the uncertainty come from?

The five methods in this lecture differ largely in which of these three kinds of uncertainty they are built to handle.

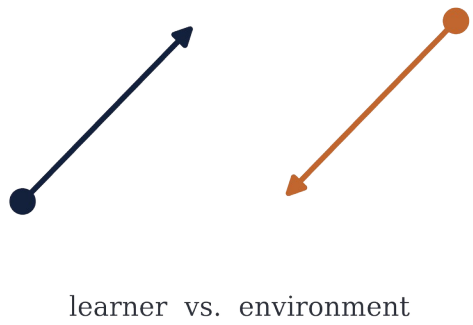
Combinatorial



Stochastic



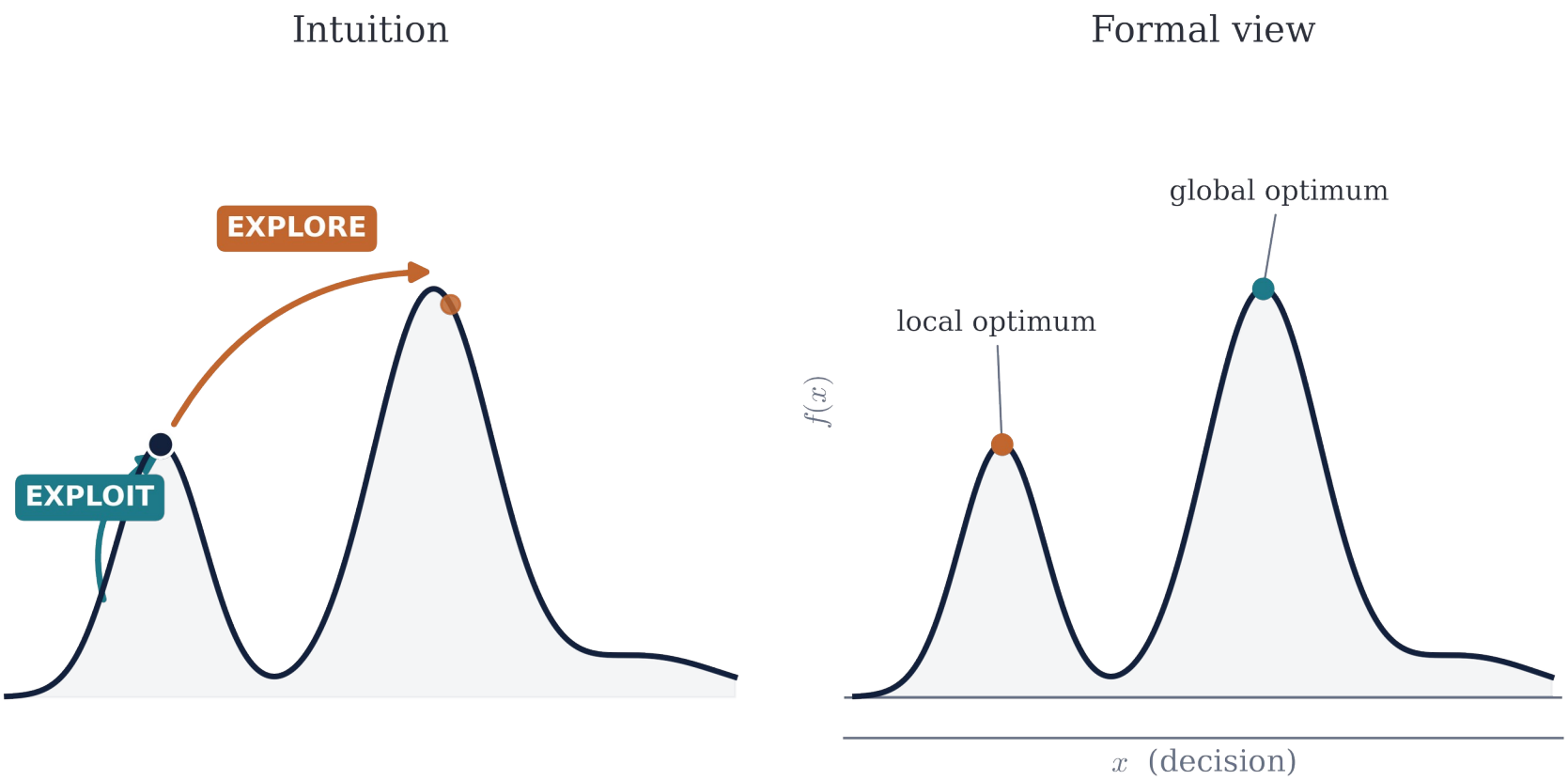
Adversarial



Combinatorial: the objective is known exactly, but the search space is too large to enumerate. **Stochastic:** outcomes are random draws from a distribution. **Adversarial:** outcomes may be shaped by a competing or worst-case force.

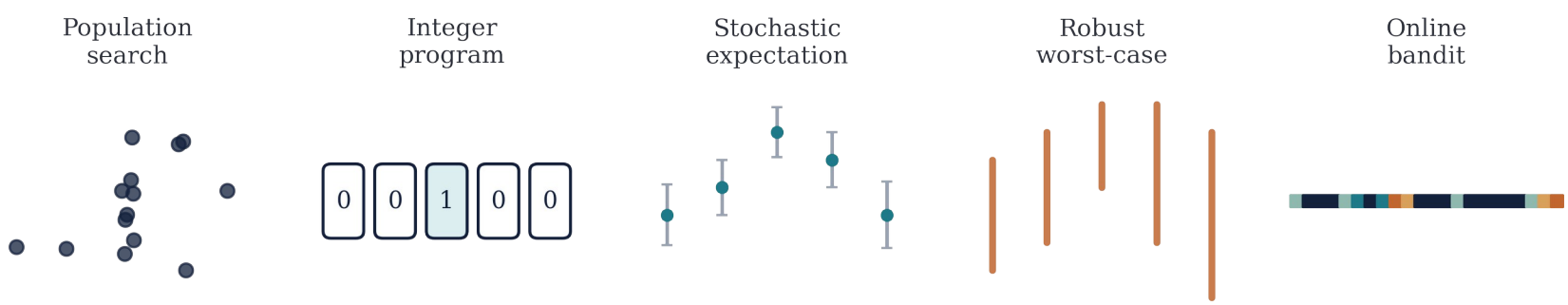
The landscape view of search

A useful mental picture for almost every method in this lecture: a surface of possible decisions, with peaks of higher and lower quality.



Exploitation climbs the nearest slope. Exploration jumps to a region that has not been evaluated yet — it might be worse, or it might be the true global optimum.

Same problem, five lenses



A **population** to evolve. A **binary vector** to optimize exactly. A **distribution** to estimate. An **interval** to guard against. A **sequence of pulls** to learn from online. We will build every one of these in detail.

PART 2 OF 8

2

Population-Based Metaheuristics

Search driven by mutation, crossover, and selection over a population of candidate solutions — the first of five lenses onto the same underlying dilemma.

EXPLORATION · EXPLOITATION

What is a metaheuristic?

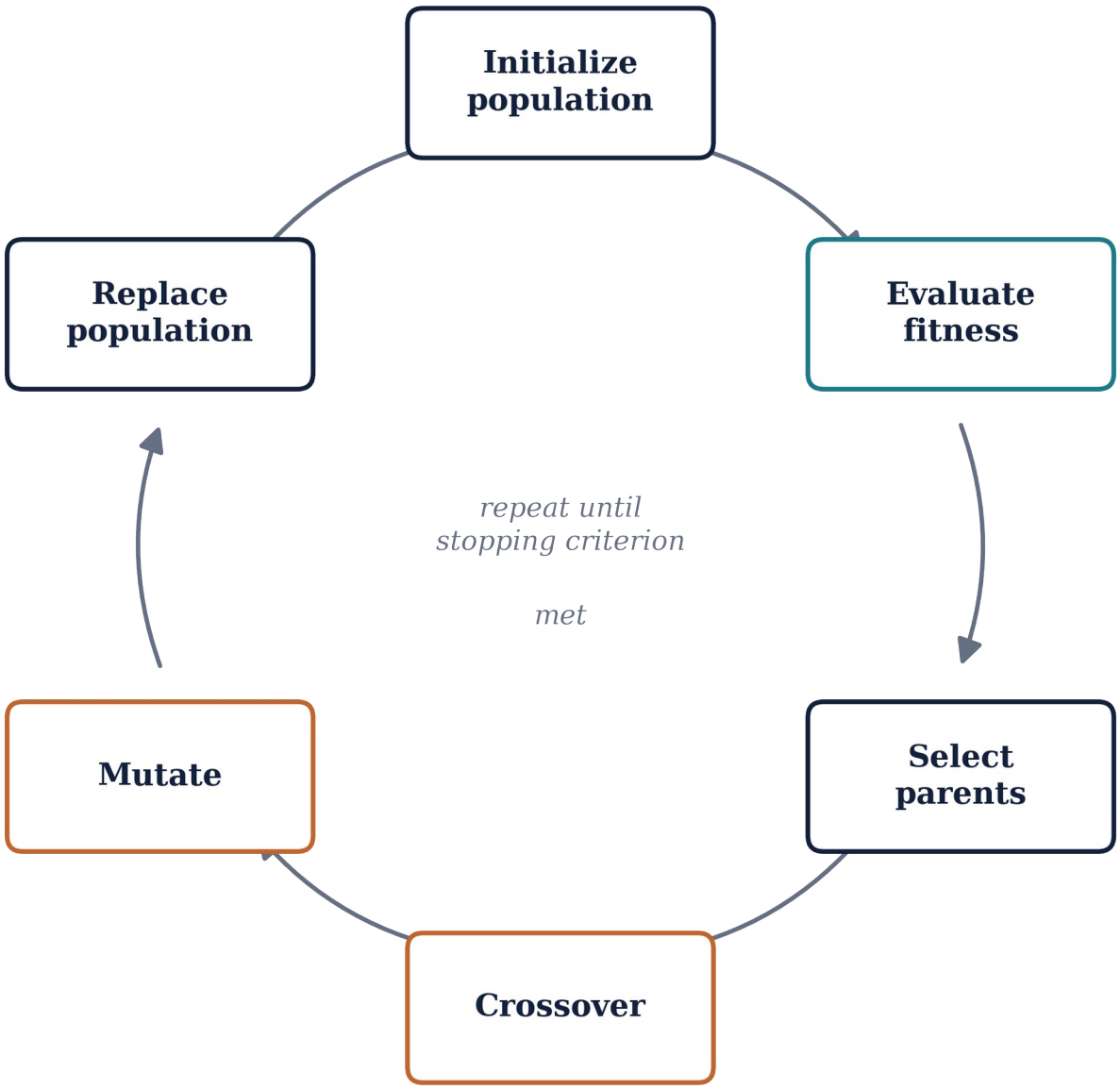
DEFINITION

A metaheuristic is a general-purpose, derivative-free search strategy that improves a population of candidate solutions through repeated cycles of variation and selection.

- Treats the objective as a **black box**: only needs to evaluate candidates, never needs gradients or convexity.
- Well suited to **discrete, combinatorial, or non-smooth** search spaces — exactly where the pricing problem lives.
- Genetic algorithms are the canonical example, and the one used throughout this section.
- No guarantee of finding the true optimum — the trade we accept for generality and simplicity.

The generational loop

*One pass through this cycle is a **generation**; a genetic algorithm repeats it until a stopping criterion is met.*



Core ingredients

FOUR INGREDIENTS, ONE LOOP

A population of candidates, a fitness function, variation operators (mutation and crossover), and a selection rule that decides who survives.

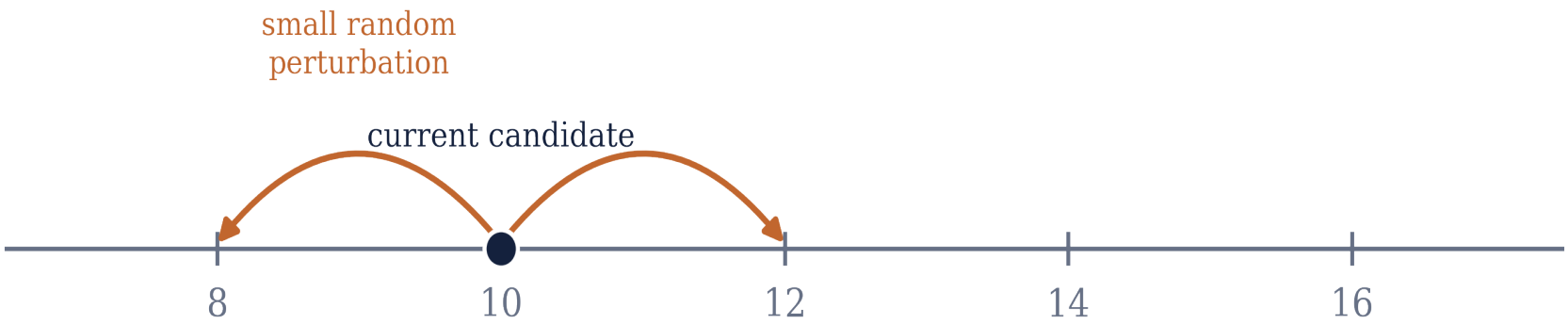
- **Population:** a multiset of candidate prices, e.g. {8, 12, 12, 14, 10, 16, …}, evaluated in parallel each generation.
- **Encoding:** each candidate is written as a short chromosome — for a 5-price menu, 3 bits suffice (000 → 8, …, 100 → 16).
- **Fitness:** the profit proxy itself — no gradient of it is ever needed.
- **Diversity:** how spread out the population is; the raw material that variation operators have to work with.

Mutation: definition and intuition

DEFINITION

Mutation perturbs a single candidate at random, independent of every other candidate in the population.

- The purest form of **exploration** in the whole method: it can move anywhere, including toward candidates selection would never have chosen.
- Typically applied with a small probability per gene, so most offspring resemble their parent closely.
- Occasionally a large or lucky mutation escapes a local optimum entirely.
- Without mutation, a genetic algorithm can only ever recombine genetic material already present — it can converge, but it can never discover.



Mutation: mechanism

$$x'_j = x_j \oplus \text{Bernoulli}(\mu) \qquad j = 1, \ldots, L$$

Bit-flip mutation with per-gene rate μ

x_j j-th gene (bit) of the chromosome

μ mutation rate — probability each gene flips

U(0,1) an independent uniform random draw per gene

- Each gene flips **independently** with probability μ, so a 3-bit chromosome typically changes 0 or 1 bits, occasionally more.
- μ close to 0: offspring almost always resemble their parent — mostly local, exploitative search.
- μ close to 0.5: offspring become essentially random — exploration overwhelms any inherited structure.

Example: mutation trajectories

A parent at price 10 spawns several mutated offspring on the profit landscape.



Most mutations are small and stay near price 10. One large mutation happens to land near price 14 — close to, but not quite at, the true optimum. **This is exploration exactly as advertised: unguided, occasionally wasteful, and occasionally decisive.**

Crossover: definition and intuition

DEFINITION

Crossover combines genetic material from two selected parents to produce one or more offspring.

- Parents are chosen **because they scored well** — so crossover is seeded by exploitation.
- But recombining two good parents can produce a genuinely new combination neither parent had — a mild form of exploration.
- This is why crossover is often described as sitting **between** pure exploration and pure exploitation.
- The building-block hypothesis: good partial solutions (“schemas”) can combine into better whole solutions.

Crossover: mechanism

$$C_1 = A_{1:k} \,\|\, B_{k+1:L} \qquad\qquad C_2 = B_{1:k} \,\|\, A_{k+1:L}$$

Single-point crossover at position k

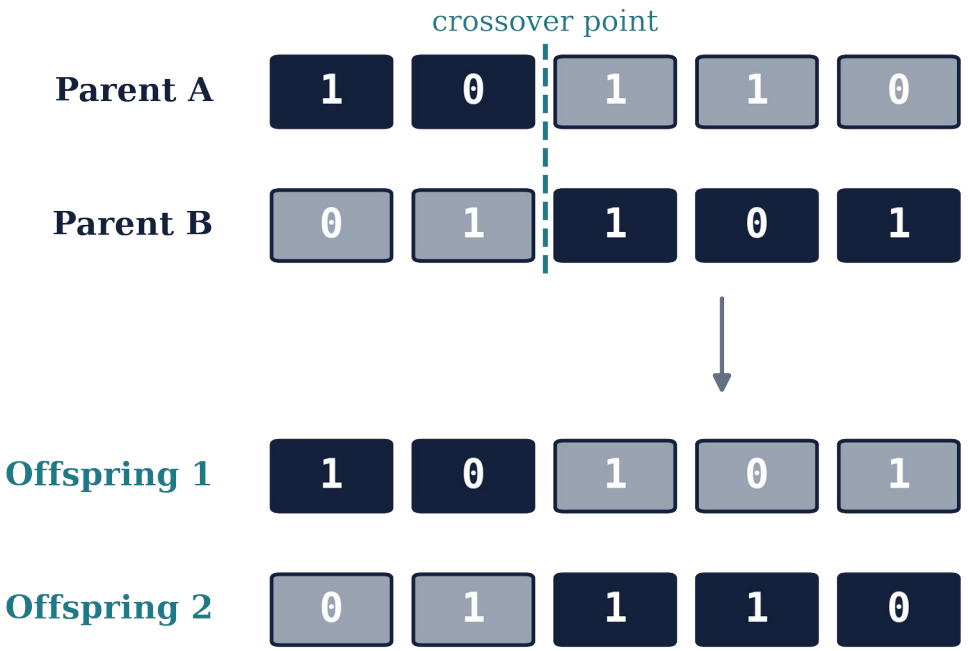
A, B the two parent chromosomes

k the crossover point, chosen at random along the chromosome

C_1, C_2 the two resulting offspring

- Everything **before** the cut comes from one parent; everything **after** comes from the other.
- Uniform crossover generalizes this: each gene independently comes from parent A or parent B.
- Only recombines material that already exists in the population — it cannot introduce a value neither parent had.

Worked example: single-point crossover



Two parent chromosomes split at the same point and swap tails, producing two new offspring that inherit structure from both parents.

How the two operators interact

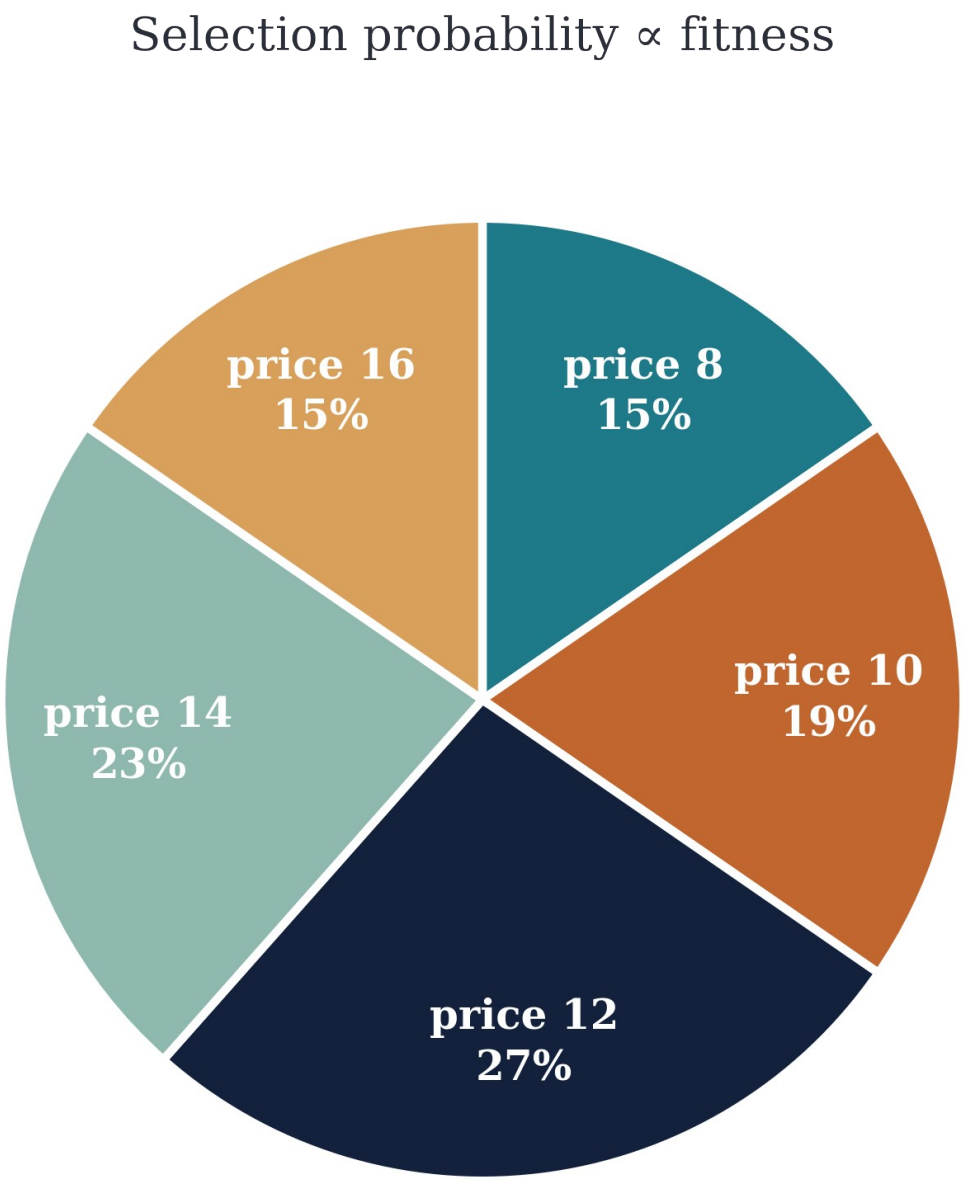
The original framing already treats mutation as exploration and crossover as a mixture — here is how they combine in a single generation.



Crossover **exploits** existing good building blocks by recombining them; mutation then **explores** just beyond what recombination alone could ever reach. Neither operator alone is sufficient — crossover without mutation can only shuffle a fixed gene pool, and mutation without crossover discards the head start that fit parents provide.

Selection: turning fitness into survival

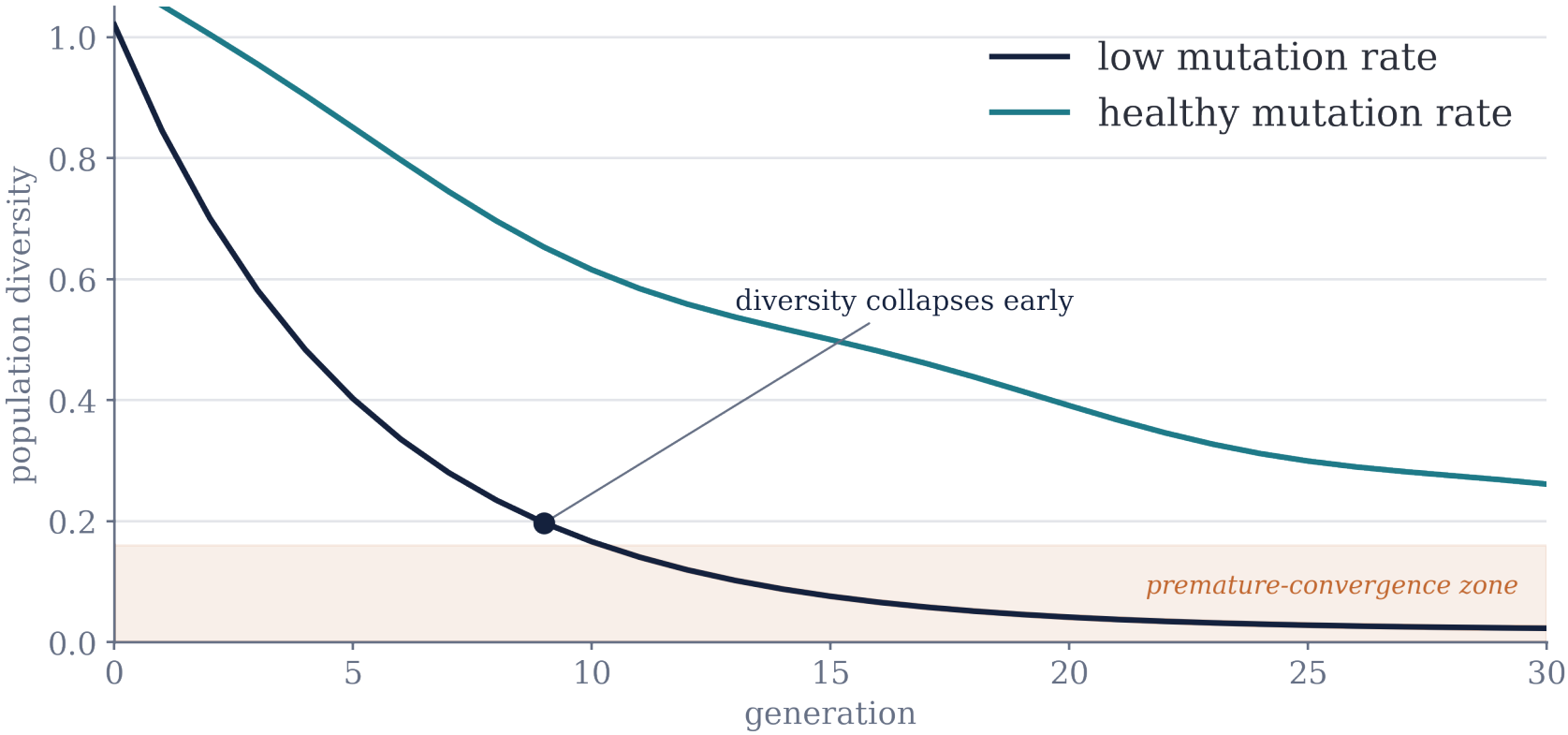
*Selection is what makes the loop **improve** rather than just wander — it is the population's exploitation mechanism.*



Roulette-wheel selection: each candidate's chance of becoming a parent is proportional to its fitness. Price 12 (profit 7) is nearly twice as likely to be selected as price 8 or price 16.

Population diversity and premature convergence

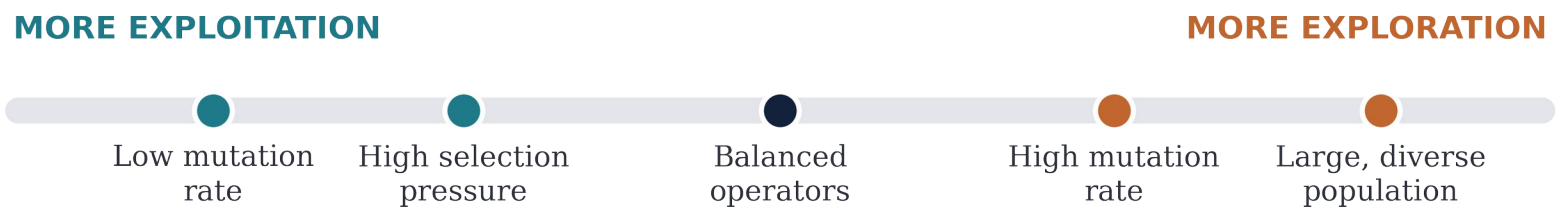
Diversity is the population's exploration budget — once it is gone, mutation is the only source of new information left.



Premature convergence: the whole population collapses onto one candidate before the true optimum has been found, and stays there — the metaheuristic equivalent of “getting stuck.”

Exploration vs. exploitation through parameter choices

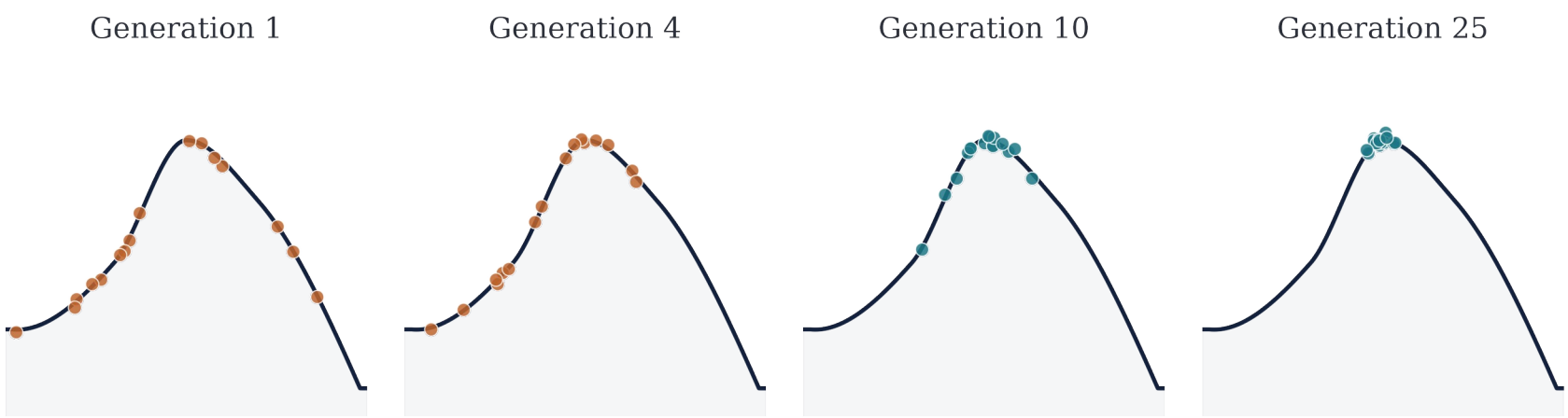
Every hyperparameter of a genetic algorithm can be read as a dial between the two extremes.



Population size also matters: a **larger population** samples more of the landscape per generation (more exploration) at proportionally higher computational cost per generation.

Watching a population converge

The full run on the toy pricing problem: 16 candidates, evolving under mutation, crossover, and selection.



By generation 25 the population has **exploited** its way to price 12 — but it only found that peak because early generations kept enough **exploratory** spread to discover it in the first place.

Assumptions, strengths, and weaknesses

? Assumptions

- Fitness can be evaluated for any candidate
- No structure (e.g. convexity) is required or exploited
- Enough generations are affordable to let selection pressure act

+ Strengths

- Simple to implement; needs no gradients
- Naturally handles discrete, non-smooth spaces
- Easy to parallelize across a population

! Weaknesses

- No optimality certificate, ever
- Many hyperparameters to tune by trial and error
- Can waste evaluations rediscovering the same region

Failure modes and computational notes

! Typical failure modes

- Premature convergence to a mediocre local optimum
- Mutation rate too high: fitness gains never stabilize
- Selection pressure too weak: effectively random search

$ Computational characteristics

- Cost ≈ population size × number of generations × one evaluation
- Evaluations dominate cost — operators themselves are cheap
- Embarrassingly parallel across the population each generation

TAKEAWAY

What metaheuristics teach us

“A population does not need any single member to be smart — it needs enough diversity to keep discovering, and enough selection pressure to keep improving.”

Exploration lives in mutation (and partly in crossover); exploitation lives in selection. The trade-off is set indirectly, through population-level parameters, rather than through any single explicit choice.

PART 3 OF 8

3

Exact Optimization & Branch-and-Bound

The same pricing decision, written as an integer program — and solved with a provable guarantee of optimality, without ever checking every candidate.

EXPLORATION · EXPLOITATION

The integer-programming formulation

$$\max_i \sum_i p_i\, x_i \qquad \text{s.t.} \quad \sum_i x_i = 1, \;\; x_i \in \{0, 1\}$$

Choosing exactly one price as a 0/1 selection problem

p_i known profit of candidate price i

x_i 1 if price i is chosen, 0 otherwise

- The objective and constraints are known **exactly** — no randomness, no adversary, just combinatorics.
- For 5 candidates this is trivial; real pricing, routing, and scheduling problems have millions of binary variables.
- Exactness is the whole point: the method searches for a **certificate of optimality**, not just a good answer.

exactly one x_i may equal 1 : $\sum_i x_i = 1$

$x =$

x for $8	x for $10	x for $12	x for $14	x for $16
0	0	1	0	0

Exploring the feasible set, exploiting bounds

REFRAMING THE DILEMMA

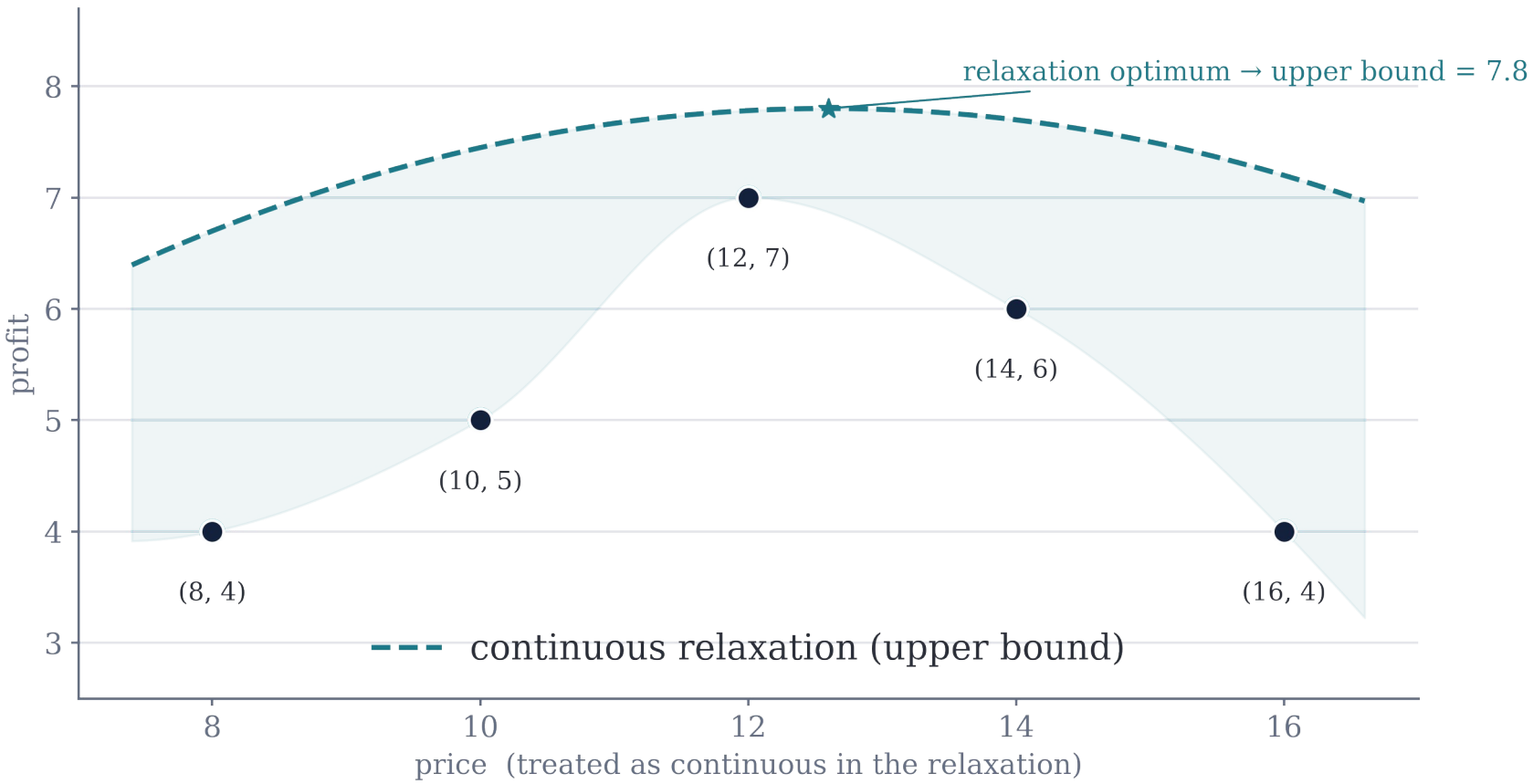
Exact methods explore by splitting the feasible set into smaller pieces, and exploit by using a bound to rule out pieces without ever inspecting them individually.

- **Exploration = branching.** Dividing the candidate set into disjoint sub-cases is how the method looks around.
- **Exploitation = bounding and pruning.** A cheap bound on each sub-case decides whether it is even worth exploring further.
- Unlike a metaheuristic, nothing here is random — every decision is a deterministic, defensible comparison of numbers.
- The prize for this discipline is a **proof**: when the search ends, we know the answer is optimal, not just good.

RELAXATION

Relaxations: a fast, optimistic stand-in

Relax the requirement that price be one of five fixed points, and allow it to vary continuously — the resulting curve gives a bound on every discrete candidate at once.



The smooth concave curve sits **above** every sampled point by construction, so its maximum (7.8) is a valid **upper bound** on the profit of any price — including ones not yet evaluated exactly.

RELAXATION

Why relaxations give valid bounds

$$x_i \in \{0, 1\} \longrightarrow x_i \in [0, 1]$$

Loosening an integrality constraint into a continuous one

- Every feasible solution to the original (integer) problem is **still feasible** in the relaxed problem.
- So the relaxed problem's optimum can only be **as good or better** than the true optimum — never worse.
- That one-directional guarantee is all branch-and-bound needs: a fast-to-compute number that never underestimates.
- The tighter the relaxation, the faster the pruning — this is an active area of research in its own right.

The mechanism, step by step

1. **Bound**
   Compute a fast upper bound on the best possible profit within the current candidate set.

2. **Compare**
   If that bound cannot beat the best solution found so far (the incumbent), stop — prune this branch.

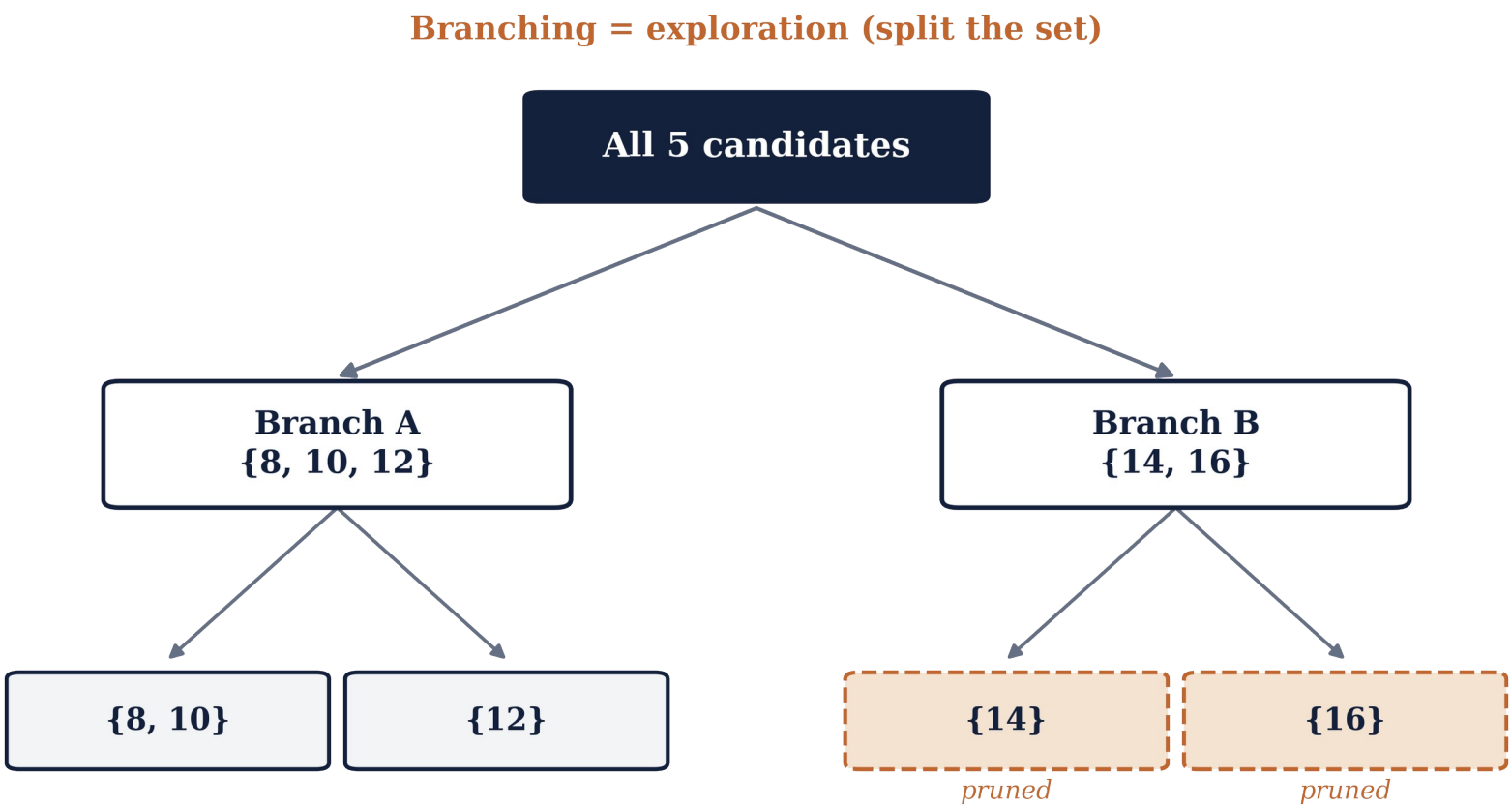
3. **Branch**
   Otherwise, split the candidate set into smaller, disjoint sub-cases.

4. **Recurse**
   Apply the same three steps to each sub-case, updating the incumbent whenever an exact evaluation beats it.

TERMINATION

The search ends when every branch has either been fully evaluated or pruned. What remains is not just a good answer — it is a **proof** that nothing better exists.

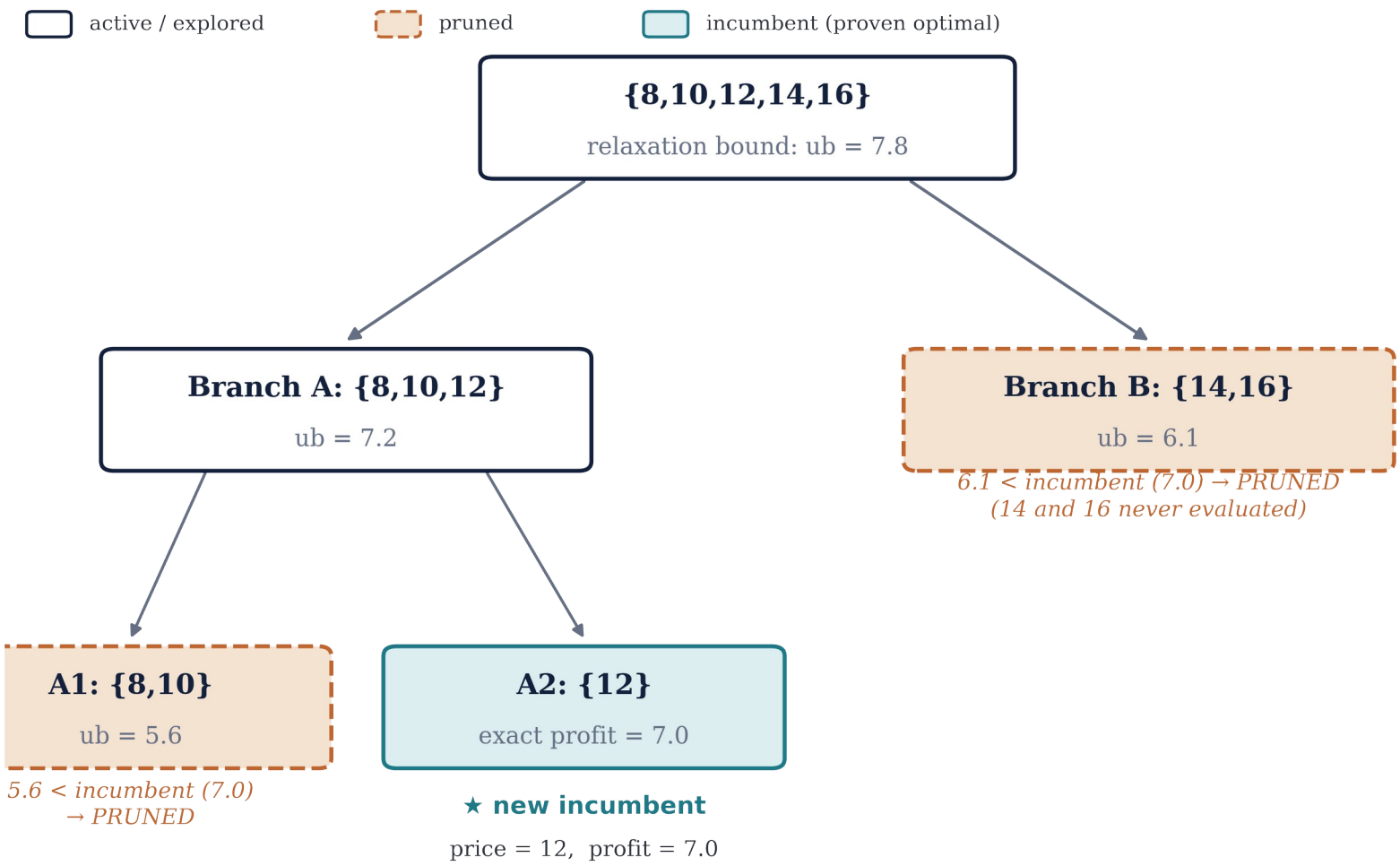
The search tree: intuition



Branching **explores** by dividing the set; a bound at each node decides — without full evaluation — whether that branch is worth **exploiting** further.

A complete branch-and-bound run

The same five prices, solved to a proof of optimality.



Only 1 of 5 candidates was ever evaluated exactly — the other 4 were ruled out by bound comparisons alone.

Reading the tree

WHAT JUST HAPPENED

Price 12 was proven optimal after exactly evaluating one candidate — the other four were eliminated by comparing bounds alone.

- The root relaxation (ub = 7.8) is loose, so the set is **branched** into A = {8,10,12} and B = {14,16}.
- Evaluating {12} exactly gives profit 7.0, which instantly becomes the **incumbent**.
- Branch B's bound (6.1) cannot beat 7.0, so it is **pruned** — prices 14 and 16 are never checked individually.
- The same logic prunes {8,10}. What remains is a certificate: **no price could have done better than 12.**

Assumptions and computational reality

? Assumptions

- The objective and constraints are known exactly, with no noise
- A valid, computable bound (relaxation) exists
- The feasible set can be split into finitely many pieces

+ Strengths

- A provable optimality guarantee, not just a good answer
- Often far faster in practice than full enumeration
- Bound quality, not problem size alone, drives performance

! Weaknesses

- Worst case is still exponential in problem size
- Needs an exactly known, noise-free objective
- Loose bounds can degrade toward brute-force enumeration

Why this is systematic, not exhaustive

THE KEY INSIGHT

Branch-and-bound never inspects most of the search space — it reasons about whole regions at once, using bounds as a substitute for evaluation.

- Every pruned branch represents **potentially many candidates ruled out in a single comparison.**
- This is exploration with a purpose: branch only where the bound says something might still be found.
- Contrast with metaheuristics: no randomness, no diversity metric — just deterministic logic and a running incumbent.
- On large problems, the fraction of the tree ever visited can be a tiny sliver of the full space.

Failure modes and computational characteristics

Typical failure modes

- Bounds too loose → almost nothing gets pruned
- Poor branch ordering → slow to find a strong incumbent
- Problem structure destroyed by a naive formulation

Computational notes

- Runtime is dominated by number of nodes explored, not evaluated
- A strong early incumbent (e.g. from a metaheuristic) prunes faster
- Memory can also become a bottleneck on very large trees

Systematic search vs. heuristic search

COMPARED TO METAHEURISTICS

Both methods search a large space by generating and discarding candidates — but they disagree completely on how much certainty that process needs to provide.

- A metaheuristic's “selection pressure” and branch-and-bound's “pruning” play the same structural role: both discard unpromising regions.
- The difference is the **currency of exploration**: a metaheuristic risks randomness for speed; branch-and-bound spends computation for certainty.
- In practice, the two are often combined: a metaheuristic supplies a strong early incumbent that lets branch-and-bound prune more aggressively.

TAKEAWAY

What exact optimization teaches us

“You do not need to look everywhere to know you have found the best answer — you only need a way to prove, cheaply, that everywhere else is worse.”

Exploration and exploitation are both deterministic here: branching explores, bounding exploits. The reward for that discipline is a guarantee no other method in this lecture can offer.

4

PART 4 OF 8

Stochastic Optimization & Approximation

What happens when profit itself is uncertain — and the only way to learn the objective is to sample it.

EXPLORATION · EXPLOITATION

Deterministic vs. stochastic objectives

THE SHIFT IN ASSUMPTION

Instead of a known profit for each price, we now assume profit depends on random demand — the objective itself is a random variable.

- Exact optimization assumed p_i was a fixed, known number for every price.
- Here, profit = g(price, demand), and **demand is random** — we only ever see realizations of it.
- The natural objective becomes the **expected** profit, not the profit itself.
- This single change is what separates every method in this section from the previous one.

Expected-value optimization

$$\max_{p} \mathbb{E}_{D}\left[\, g(p, D) \,\right]$$

Maximize expected profit over the demand distribution

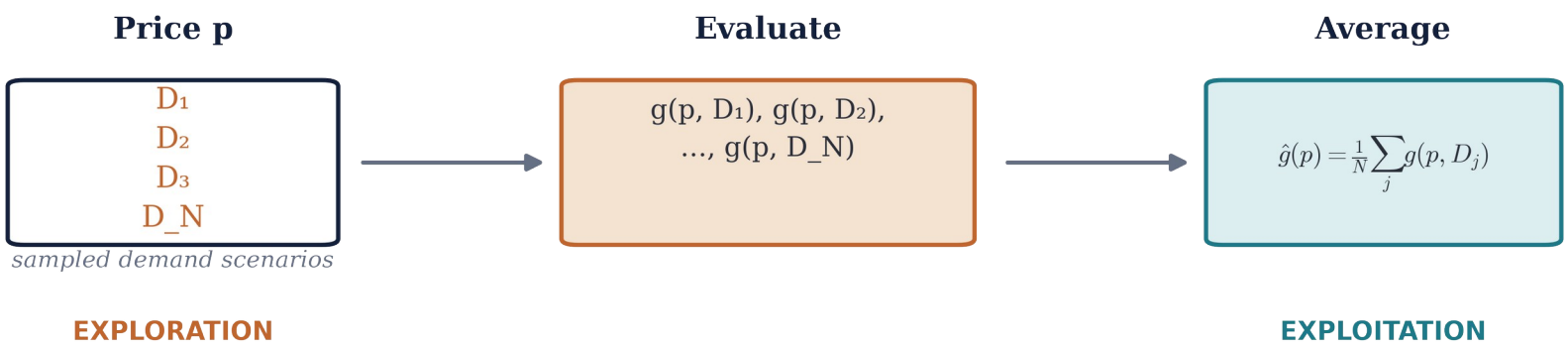
$\boldsymbol{D}$ random demand, drawn from an unknown distribution

$\boldsymbol{g(p, D)}$ realized profit at price p under demand D

$\mathbb{N}[\cdot]$ expectation over the randomness in D

- The expectation itself typically **cannot be computed exactly** — the demand distribution is rarely known in closed form.
- So it must be **estimated**, which means it must be **sampled**.
- This is the seed of the explore/exploit story for the whole section.

Turning samples into an estimate



Sampling demand scenarios is exploration — it is how the method learns anything at all about an unknown distribution. **Averaging and optimizing the estimate is exploitation** — it is how that information gets turned into a decision.

The SAA estimator

$$\hat{g}_N(p) = \frac{1}{N}\sum_{j=1}^{N} g(p, D_j) \qquad D_j \overset{iid}{\sim} \mathcal{D}$$

The sample-average approximation of expected profit

N number of demand scenarios sampled

D_j the j-th independently drawn demand scenario

$\check{g}_N(p)$ the sample-average estimate of expected profit at price p

- By the law of large numbers, ğ_N(p) → the true expected profit as N → ∞.
- For finite N, ğ_N(p) is only an **estimate** — it carries sampling error that shrinks like 1/√N.
- The optimization step then simply picks the price with the highest ğ_N(p).

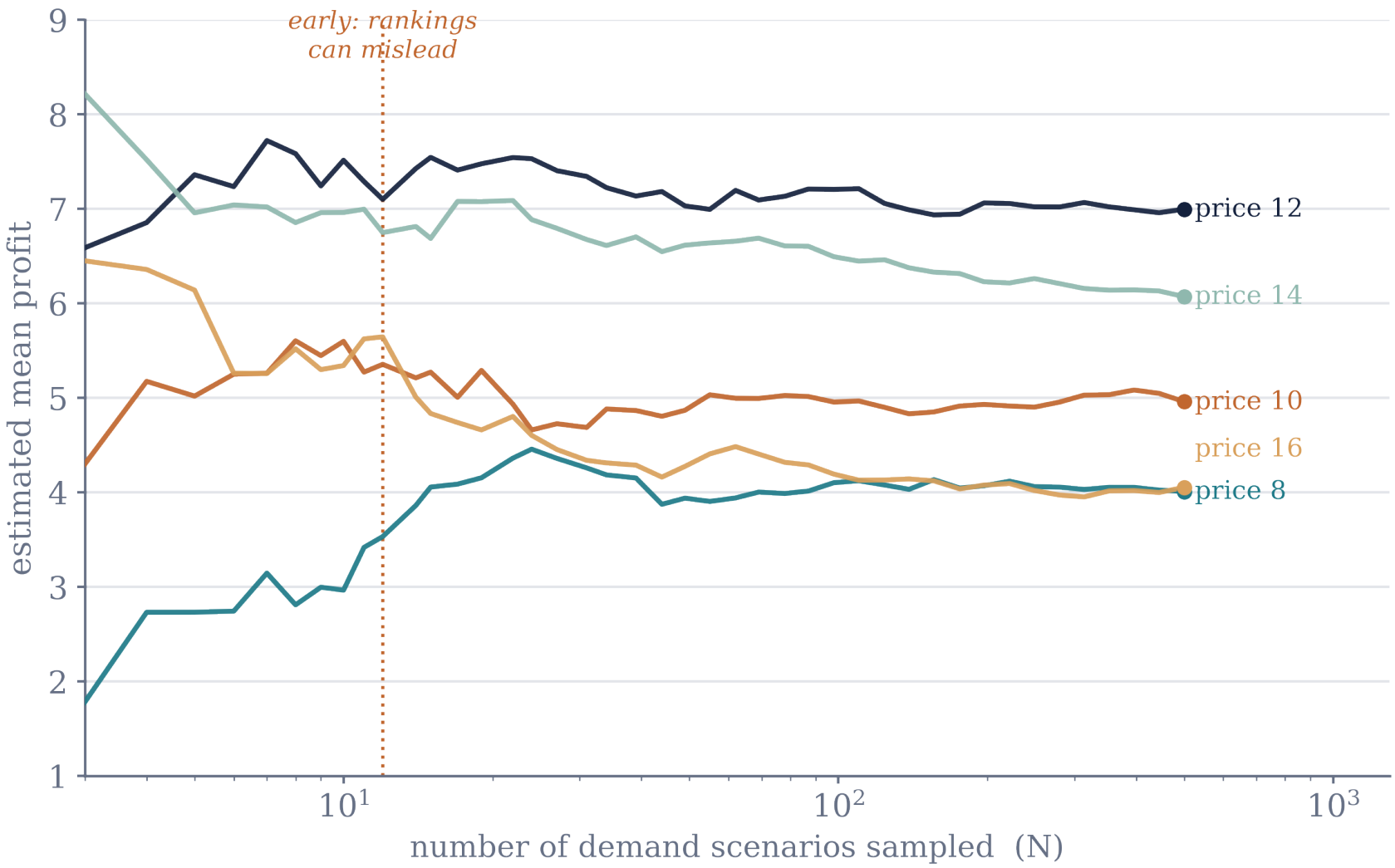
Profit across sampled demand scenarios

Price	Low demand	Medium demand	High demand	Sample mean
8	2	4	6	4.0
10	3	5	7	5.0
12	4	7	10	7.0
14	3	6	9	6.0
16	1	4	7	4.0

With only 3 scenarios, this table **is** the entire estimate — and it happens to recover the true ranking. With fewer or noisier scenarios, it easily would not.

How the estimate evolves as N grows

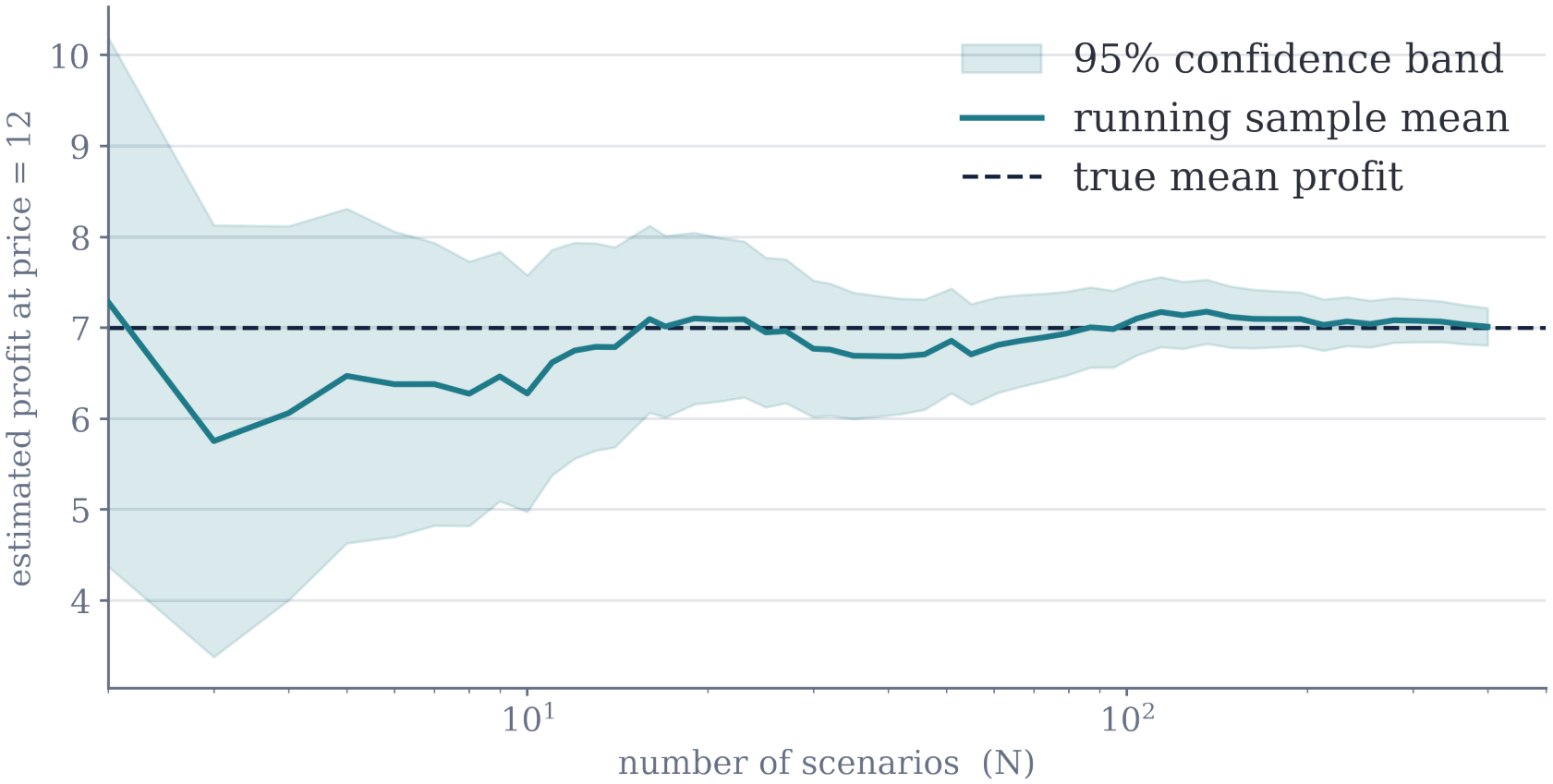
All five prices, estimated simultaneously from the same growing stream of demand scenarios.



Early on, the estimated ranking of prices is essentially noise — price 14 briefly looks competitive with price 12. By N ≈ 100, the ranking has **stabilized** on the true optimum.

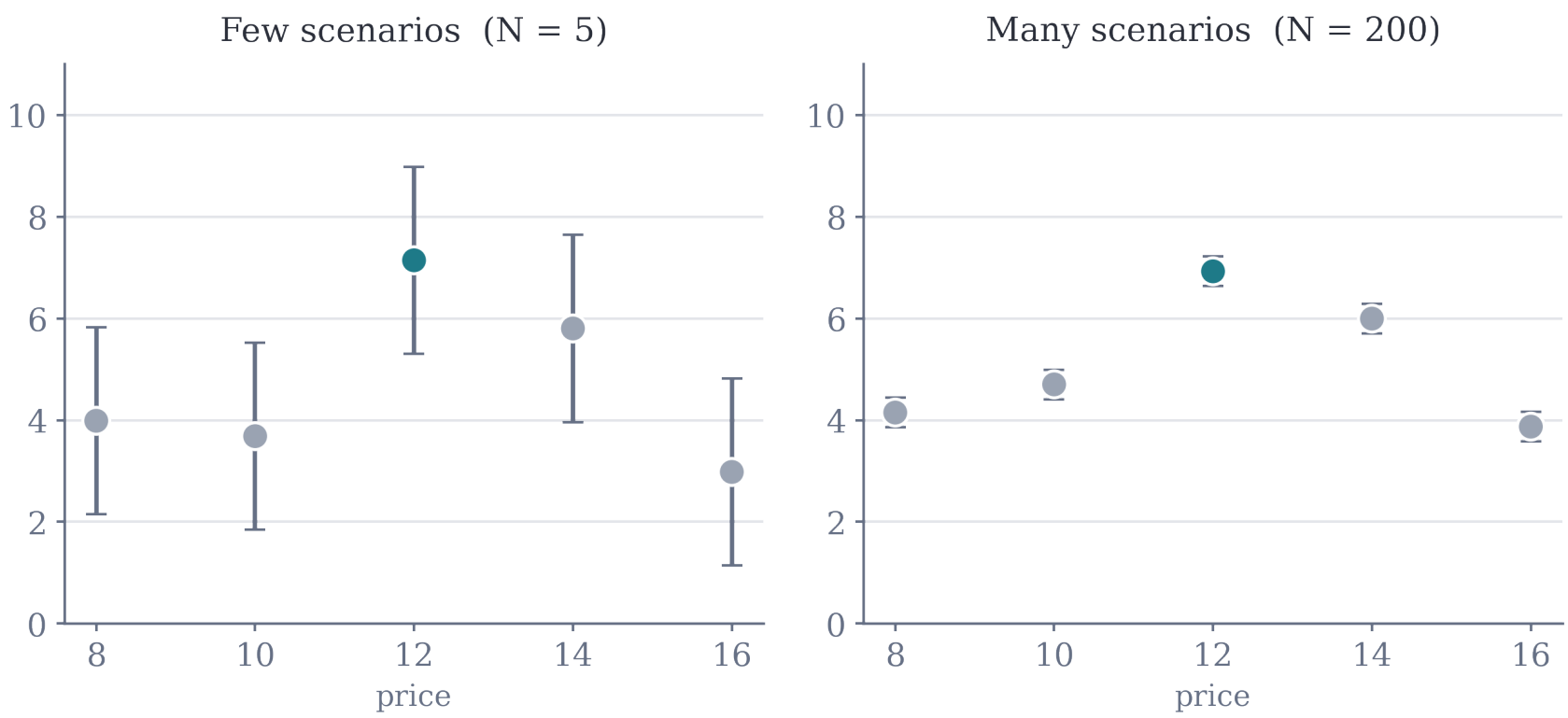
A closer look at one price

The running estimate for price 12, with a shrinking 95% confidence band.



The confidence band's width shrinks proportionally to 1/√N — doubling precision requires **four times** as many scenarios, not two.

Why sampling error matters for decisions



With N = 5, price 14's confidence interval overlaps price 12's — the “best” price is not yet statistically distinguishable from its neighbor. With N = 200, the ranking is unambiguous.

Exploration is data collection

THE EXPLICIT MAPPING

In this method, exploration and exploitation are not woven together — they are two separate, sequential phases.

- **Exploration = sampling.** Every additional demand scenario is a small, unbiased investment in reducing uncertainty.
- **Exploitation = optimizing the estimate.** Once scenarios are collected, the decision itself is deterministic — just pick the arg max.
- Unlike a bandit, this method does not have to act while it learns: sampling costs are usually offline (e.g., simulation, historical data, market research), not experiments on real customers.

Assumptions, strengths, weaknesses

? Assumptions

- Demand scenarios can be sampled (simulated, historical, or forecasted)
- Scenarios are representative of the true distribution
- The sample average is a sensible proxy for the true expectation

+ Strengths

- Turns an intractable expectation into ordinary optimization
- Error shrinks predictably and quantifiably with more data
- Works for almost any simulate-able uncertainty

! Weaknesses

- Needs enough samples for reliable ranking near-ties
- Biased scenario generation biases the whole answer
- Says nothing about worst-case outcomes

Failure modes and computational notes

Typical failure modes

- Too few scenarios → a noisy price looks best by chance
- Scenario generator does not match reality → confidently wrong
- Rare but high-impact scenarios under-sampled

Computational notes

- Cost scales with N × (cost of evaluating one scenario)
- Embarrassingly parallel across scenarios
- Precision improves slowly: to halve the error, quadruple N

From certainty to estimated certainty

COMPARED TO EXACT OPTIMIZATION

Sample-average approximation turns a stochastic problem back into something that looks deterministic — at the price of accepting a margin of statistical error.

- Once scenarios are fixed, optimizing ğ_N(p) is a completely ordinary (even exact) optimization problem.
- The uncertainty has not disappeared — it has been **pushed upstream** into the sampling step.
- This is a recurring pattern: methods often convert one kind of uncertainty into a different, more tractable one.

TAKEAWAY

What stochastic optimization teaches us

“When the objective itself is uncertain, exploration and exploitation split cleanly in time: first learn what the world looks like, then optimize as if you were certain.”

The quality of the final decision is only ever as good as the sample that informed it — which is exactly the trade-off every other method in this lecture also has to make, just less visibly.

5

PART 5 OF 8

Robust Optimization

What to do when the demand distribution itself cannot be trusted — optimize for the worst case the world could plausibly present.

Guarding against uncertainty, not averaging it

DEFINITION

Robust optimization chooses the option with the best worst-case value over a set of plausible futures, rather than the best value on average.

- Stochastic optimization asks: **what happens on average?** Robust optimization asks: **what is the worst that could happen?**
- No probability distribution is required — only a set of values considered plausible.
- The result is a **guarantee**: whatever demand turns out to be, profit will be at least this much.
- This is the natural choice when the cost of a bad outcome is severe, or the distribution itself is not trustworthy.

The min-max formulation

$$\max_{p} \min_{D \in \mathcal{U}} g(p, D)$$

Maximize the worst case over the uncertainty set

$$\mathcal{U} = \left\{ D : D_{\min} \leq D \leq D_{\max} \right\}$$

A simple interval uncertainty set

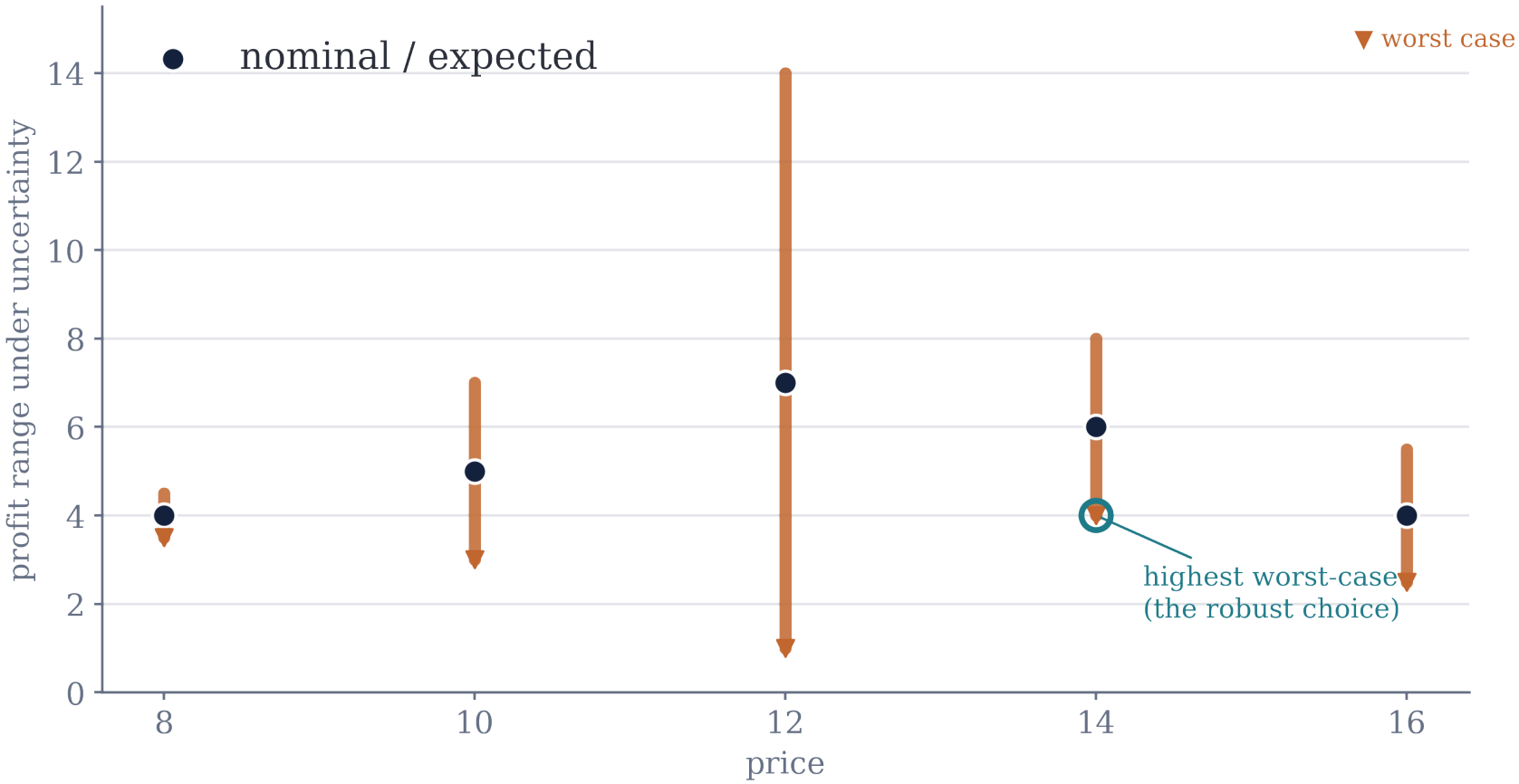
$\mathcal{U}$ the uncertainty set — every demand value considered possible

$\min_{D \in \mathcal{U}}$ the worst realization of demand, for a fixed price

- The inner minimization is an **adversary**: it picks the worst D it is allowed to, for whatever price you chose.
- The outer maximization then picks the price that performs best **against that adversary**.

What a price can plausibly earn

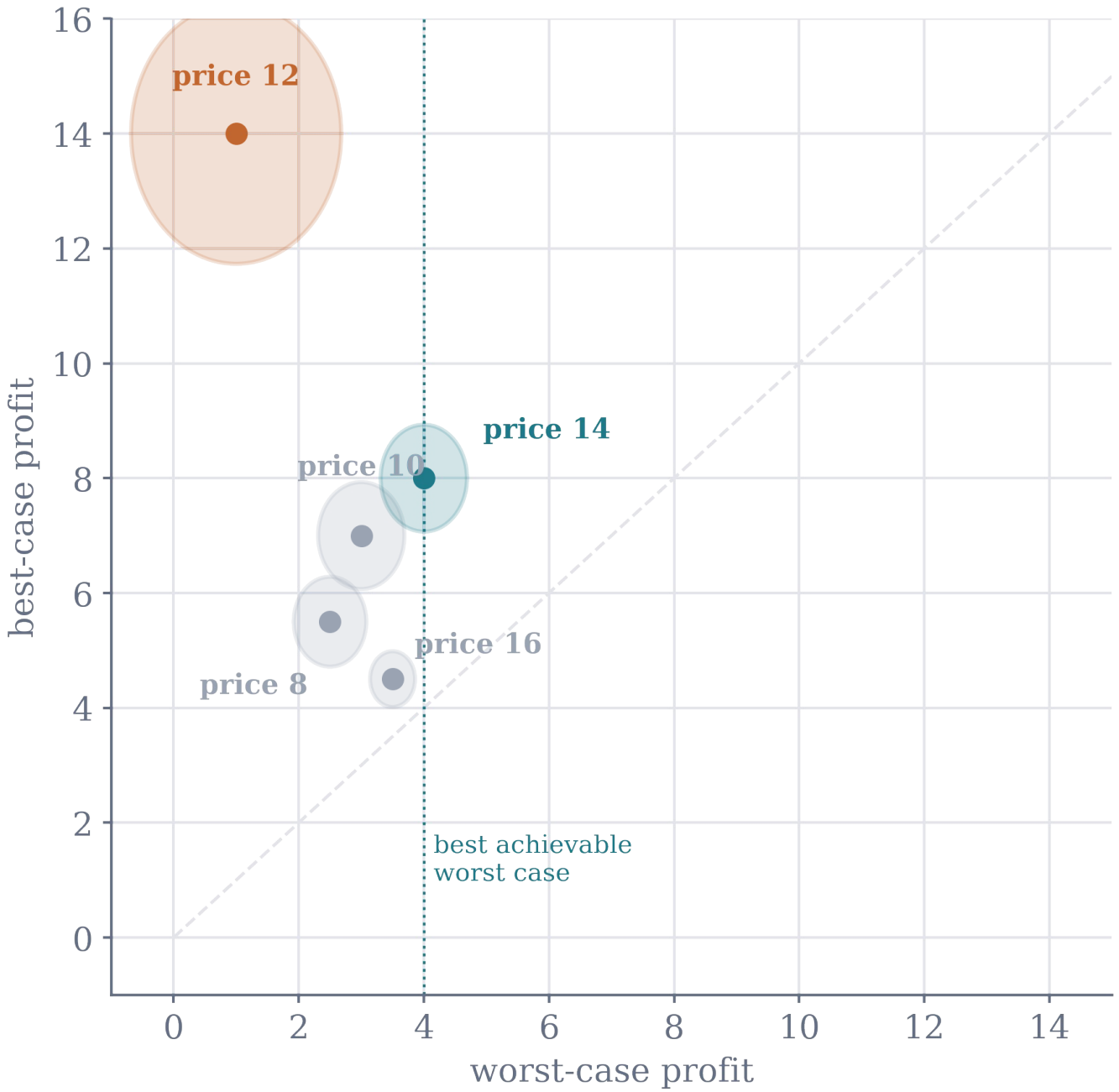
Interval uncertainty for each price: the black dot is the nominal (expected) profit; the bar spans every value considered plausible.



Exploration here means characterizing the uncertainty set itself — understanding how bad things could plausibly get for each option, before ever comparing them.

A geometric view

Each price plotted by its worst-case profit (x) and best-case profit (y) — the uncertainty set becomes a region, not just an interval.



Price 12 sits far to the **left** (a terrible worst case) despite sitting highest on the page (the best best case). Price 14 gives up some upside to sit furthest **right**.

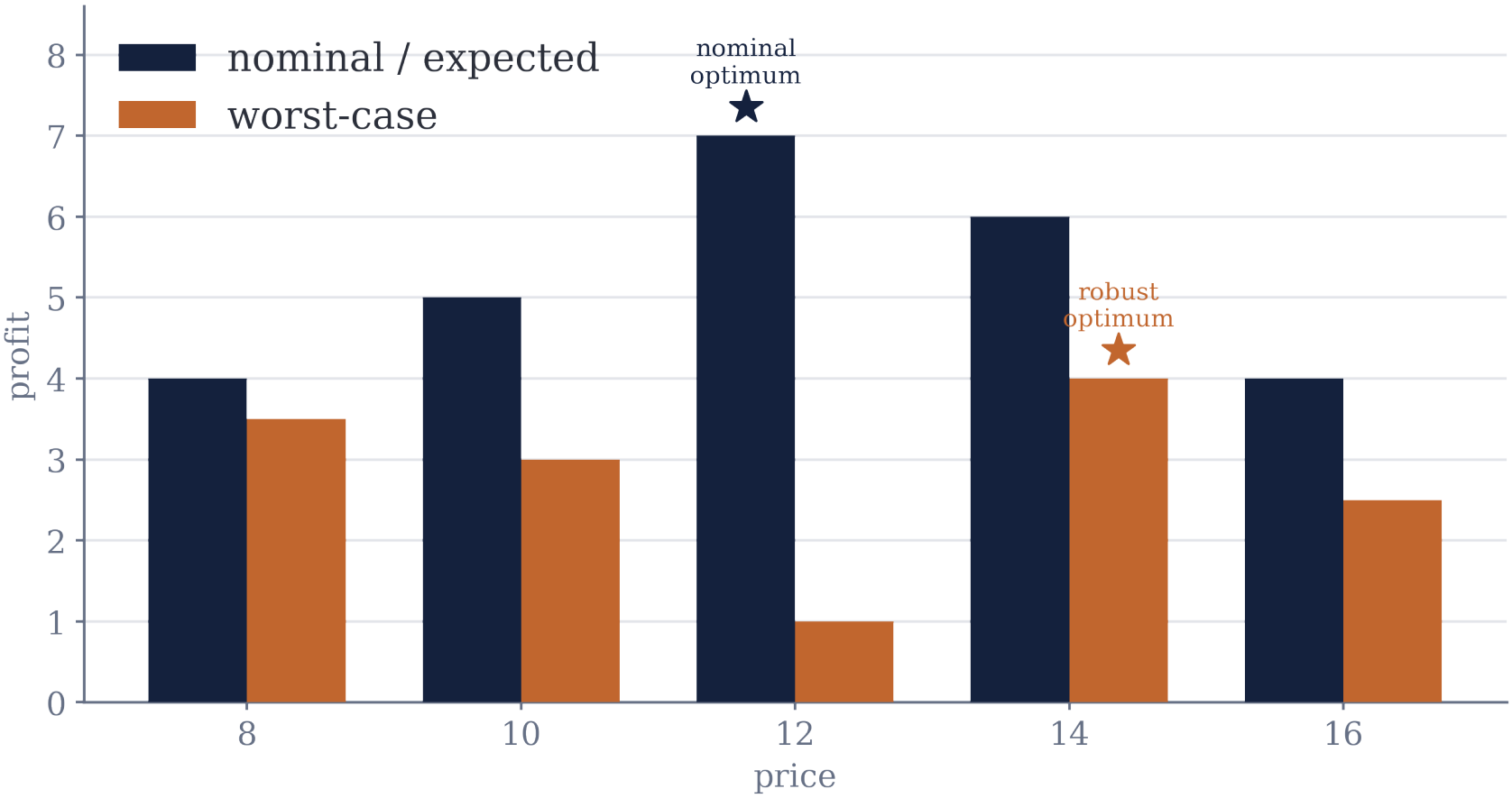
WORKED EXAMPLE

Nominal optimum vs. robust optimum

Price	8	10	12	14	16
Nominal (mean)	4.0	5.0	7.0	6.0	4.0
Worst case	3.5	3.0	1.0	4.0	2.5

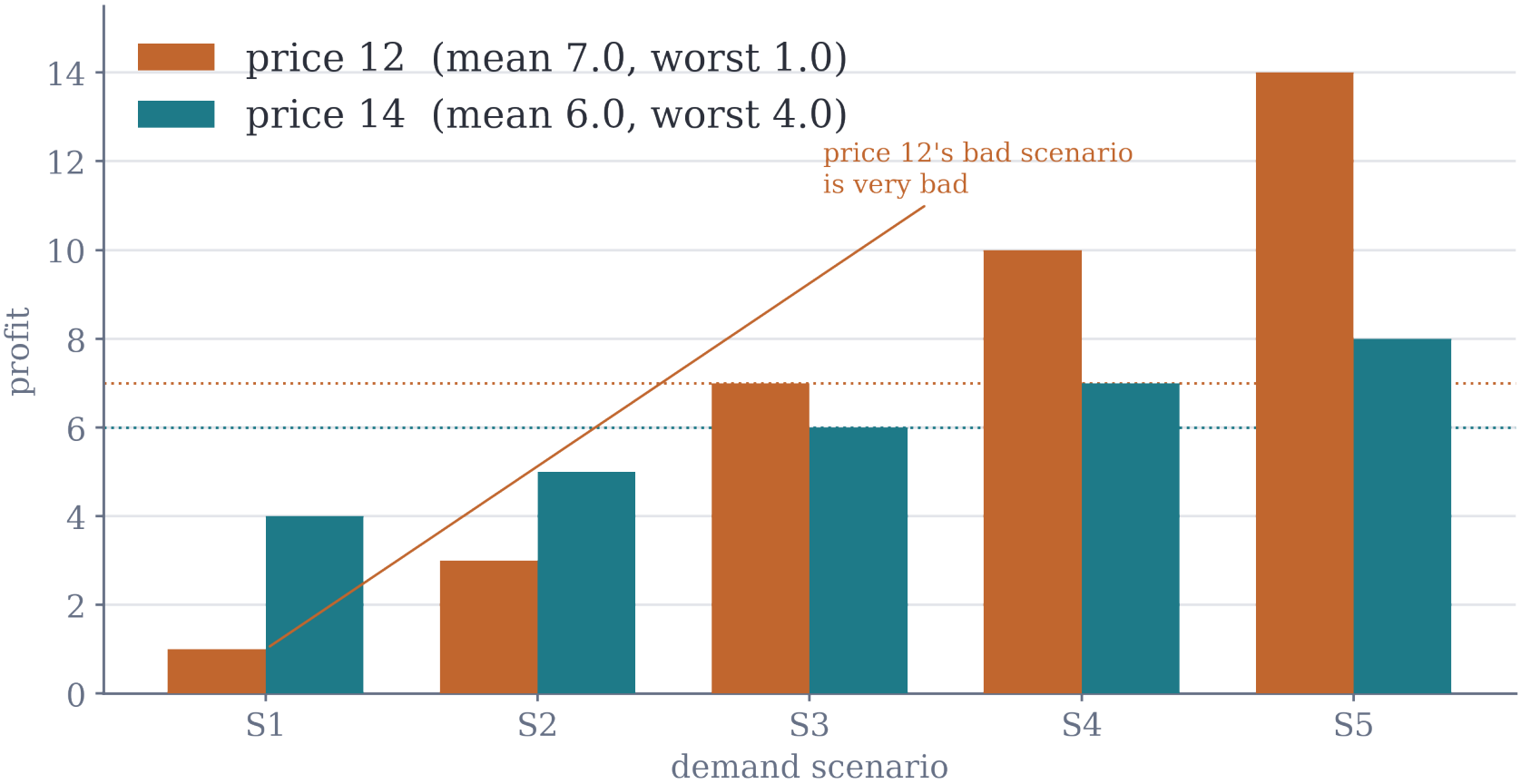
TWO RANKINGS, TWO WINNERS

Ranked by the mean, **price 12 wins**. Ranked by the worst case, **price 14 wins** — a completely different answer from the same underlying data.



When the average-best is not the robust-best

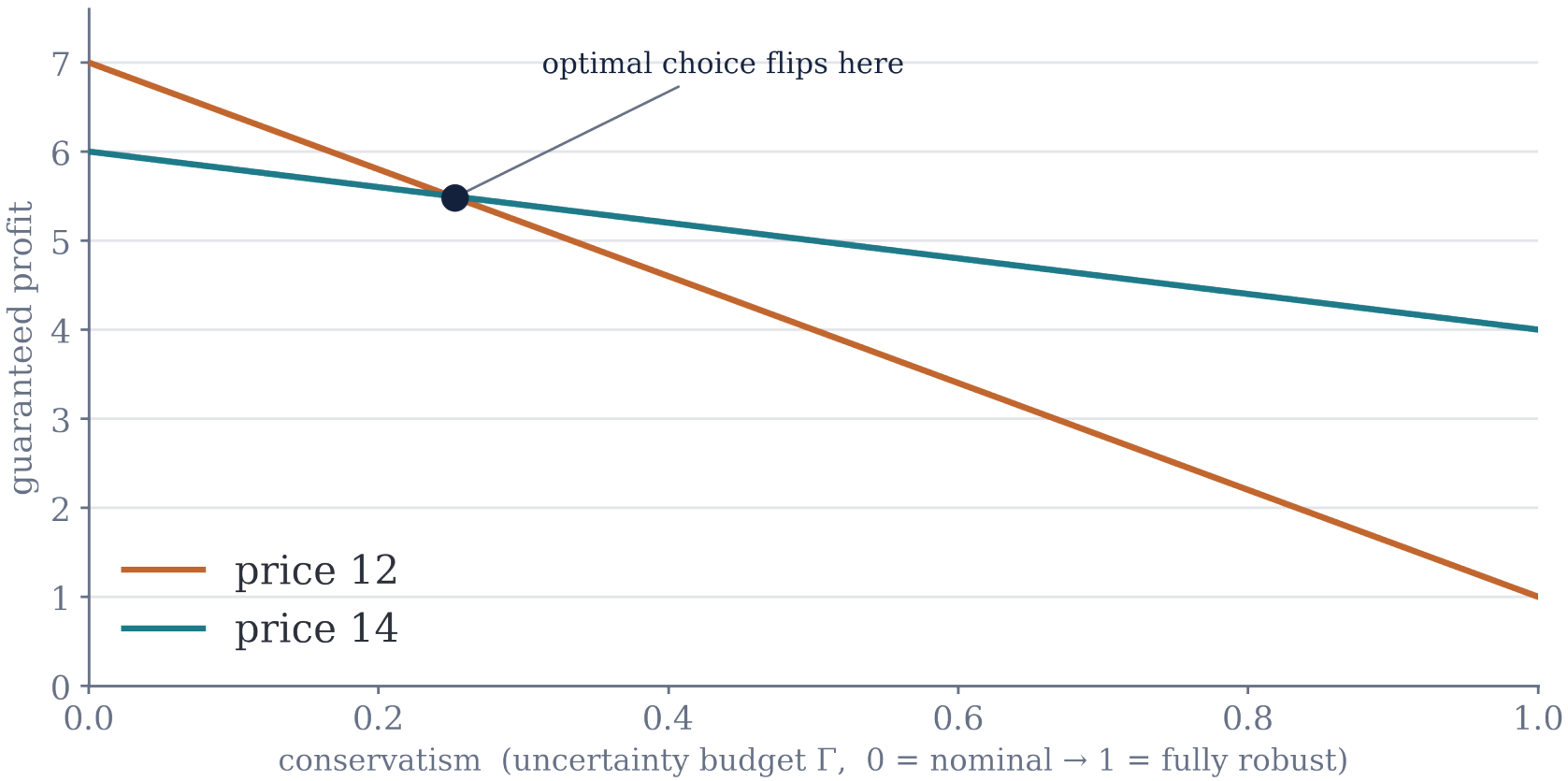
Five illustrative demand scenarios for prices 12 and 14, exposing exactly why the rankings diverge.



Price 12 wins on average **because of one spectacular scenario (S5)** that offsets one catastrophic one (S1). Price 14 never spikes — but it also never collapses.

The price of robustness

As the uncertainty budget grows — protecting against more adversarial scenarios at once — the guaranteed profit of each price changes.



At low conservatism, price 12's high mean wins. Past a threshold, price 14's steadiness wins. **The "right" answer depends on how much protection you are willing to pay for.**

Exploration vs. exploitation, reframed

THE MAPPING

There is no search process here in the usual sense — the explore/exploit split happens entirely at the level of what the method chooses to trust.

- **Exploration = characterizing ^{3}.** Deciding how wide the uncertainty set should be is itself a judgment call about what to take seriously.
- **Exploitation = solving the worst case exactly.** Once ^{3} is fixed, the min-max problem is solved with full rigor, often exactly.
- A wider ^{3} is more “exploratory” about what could go wrong — and produces a more conservative, lower-guarantee decision.

Assumptions, strengths, weaknesses

? Assumptions

- A meaningful uncertainty set can be specified
- Worst-case protection is worth its cost in lost upside
- The adversary's power (the set's size) is chosen sensibly

+ Strengths

- No probability distribution required, only bounds
- A hard guarantee, not just an expectation
- Naturally suited to high-stakes, low-tolerance decisions

! Weaknesses

- Can be needlessly conservative if ³ is too wide
- Ignores likelihood entirely — a 1-in-a-million event counts fully
- Choosing ³ itself is often more art than science

Failure modes and computational notes

Typical failure modes

- Uncertainty set too wide → paralyzed by over-caution
- Uncertainty set too narrow → false sense of security
- Ignoring correlation between uncertain quantities

Computational notes

- Simple interval sets keep the problem as easy as the nominal one
- More complex sets (ellipsoids, budgets) need specialized solvers
- Often only mildly harder than solving the nominal problem once

Optimism vs. pessimism as a spectrum

COMPARED TO STOCHASTIC OPTIMIZATION

Nominal, expected-value, and worst-case optimization are three points on a single spectrum of how much benefit of the doubt a decision-maker extends to uncertainty.

- Nominal optimization: assume the average scenario happens (maximally optimistic).
- Stochastic optimization: weigh every scenario by its probability (calibrated).
- Robust optimization: assume the worst permitted scenario happens (maximally pessimistic).
- The “budget of uncertainty” from the price-of-robustness figure lets a decision-maker sit **anywhere between these extremes.**

What robust optimization teaches us

“When the cost of being wrong is not symmetric, the best average outcome is the wrong thing to optimize — exploration means asking how bad things could get, not just how good.”

Exploitation here is not about acting on a best guess, but about acting on a guarantee. The trade-off is upside sacrificed for certainty — paid for, deliberately, up front.

PART 6 OF 8

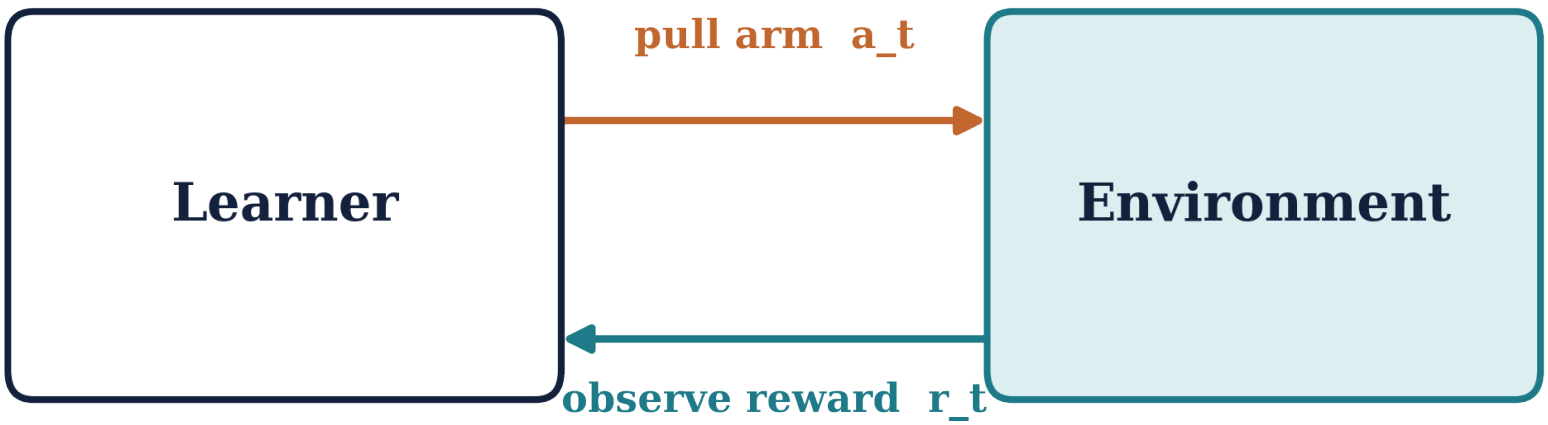
Multi-Armed Bandits

The decision is no longer one-shot: the same price is chosen again and again, and every round is a chance to learn and to earn at the same time.

EXPLORATION · EXPLOITATION

The bandit problem, set up

Each price is an "arm." Pulling it reveals a reward — and only that arm's reward.



updates its estimate
of each arm's value

generates a reward,
seen or unseen by rule

Unlike every method so far, the decision-maker acts **repeatedly**, and each action **simultaneously** produces both a payoff and a piece of information.

The classical stochastic bandit

SETUP

Each of the 5 prices has a fixed, unknown mean reward. Pulling arm i yields a noisy sample of that arm's mean, independent across rounds.

- The means themselves (4, 5, 7, 6, 4) are exactly the toy example's profit values — just unknown to the learner in advance.
- The learner does not know these numbers; it only ever sees noisy realizations, one arm at a time.
- This is a genuinely online problem: every round matters, not just the final answer.

Measuring the cost of not knowing

$$R_T = \sum_{t=1}^{T} (\mu^\star - \mu_{a_t})$$

Cumulative regret over T rounds

μ^* the mean reward of the best arm (unknown to the learner)

a_t the arm played at round t

R_T total reward given up by not always playing the best arm

- Regret is the single number every bandit algorithm in this section is trying to keep small.
- A good algorithm has regret that grows **slower and slower** over time — ideally logarithmically.



The core tension, sharpened

WHY THIS IS THE HARDEST VERSION OF THE DILEMMA

In every earlier method, exploring was free or offline. In a bandit, every single exploratory pull is also a real, scored action — exploration itself costs regret.

- There is no separate “learning phase” unless the algorithm deliberately creates one.
- Committing too early **and** exploring too long are both directly, numerically penalized.
- Every algorithm in this section is a different answer to exactly one question: **when has this arm been tried enough?**

The simplest possible strategy

1

Explore

Pull each of the 5 arms m times, in round-robin, ignoring all outcomes until this phase ends.

2

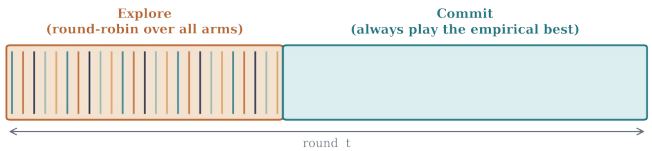
Estimate

Compute the empirical mean reward for every arm from the exploration data.

3

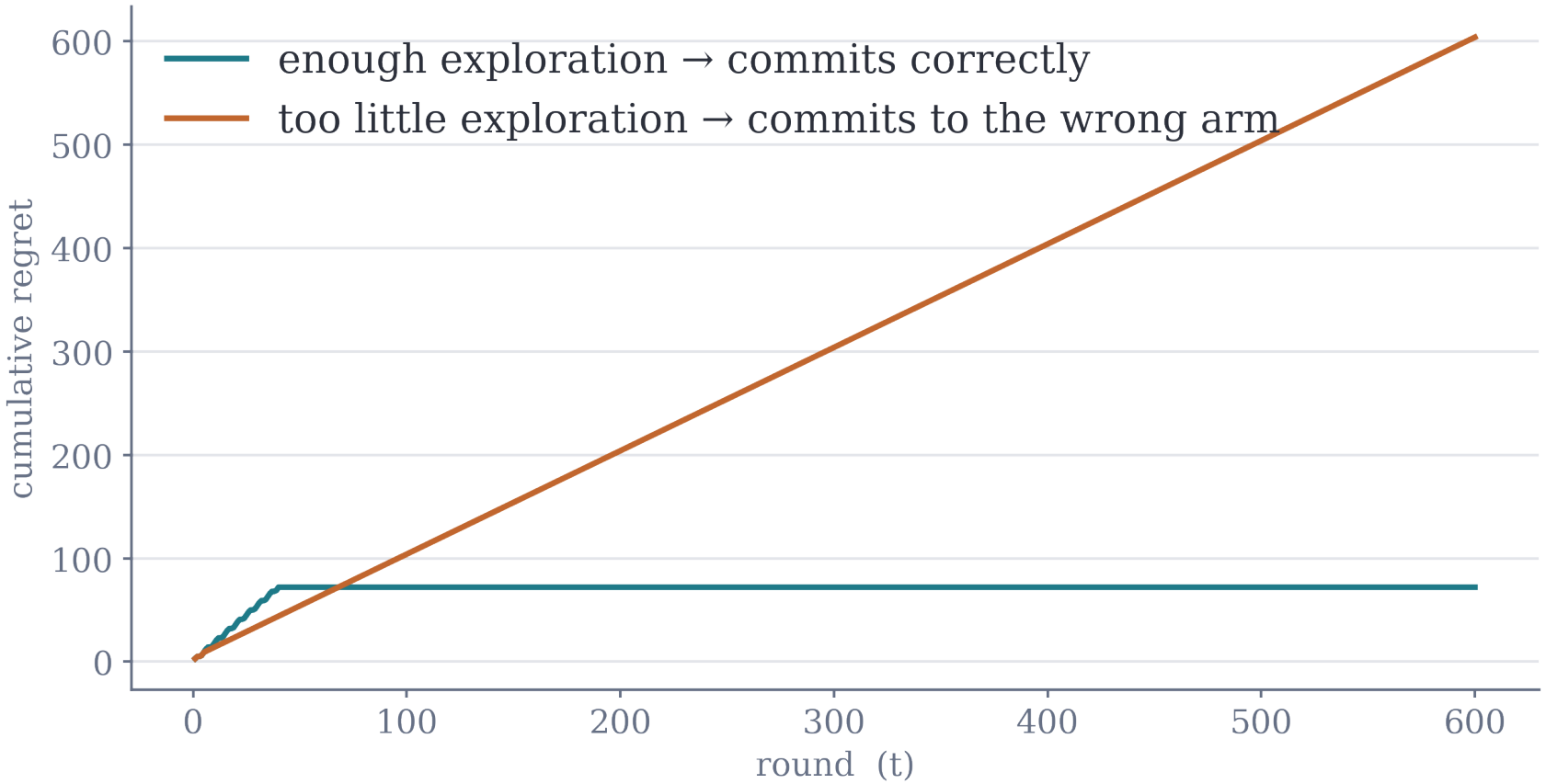
Commit

Play the arm with the largest empirical mean for every remaining round, forever.



Two hard-separated phases: all exploration first, all exploitation after.

Why the commit is risky



With enough exploration (m large), regret flattens after commit. With too little, ETC **locks in the wrong arm** and regret grows in a straight line — forever.

Assessment

+ Strengths

- Extremely simple to implement and explain
- Exploration cost is known and bounded in advance
- Easy to reason about and to audit

! Weaknesses

- m must be chosen in advance, without knowing the gaps between arms
- All-or-nothing: never revisits the decision after committing
- A single unlucky exploration phase is never corrected

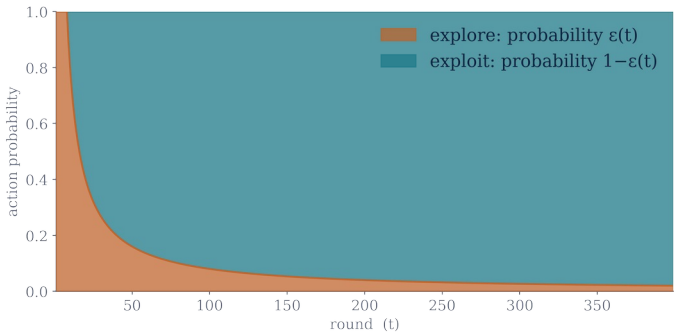
Explore and exploit, every round

1 **Flip a biased coin**
With probability ε(t), explore; with probability 1−ε(t), exploit.

2 **If exploring**
Pick an arm uniformly at random, regardless of its current estimate.

3 **If exploiting**
Pick the arm with the highest empirical mean reward so far.

4 **Decay ε**
Shrink ε(t) over time (e.g. ε(t) ≈ c/t) so exploration fades as confidence grows.



Exploration probability shrinks steadily as rounds accumulate.

Assessment

+ Strengths

- Never fully stops exploring — can recover from an early bad estimate
- One intuitive parameter to tune
- Easy to adapt online, unlike ETC's fixed commitment

! Weaknesses

- Exploration is undirected — wastes pulls on obviously-bad arms
- Decay schedule for ε(t) still needs to be chosen
- Typically dominated in theory by optimism-based methods

The optimism principle

$$\mathrm{UCB}_i(t) = \hat{\mu}_i(t) + \sqrt{\frac{2\log t}{N_i(t)}}$$

Upper Confidence Bound for arm i at round t

$\hat{\mu}_i(t)$ empirical mean reward of arm i so far

$N_i(t)$ number of times arm i has been pulled

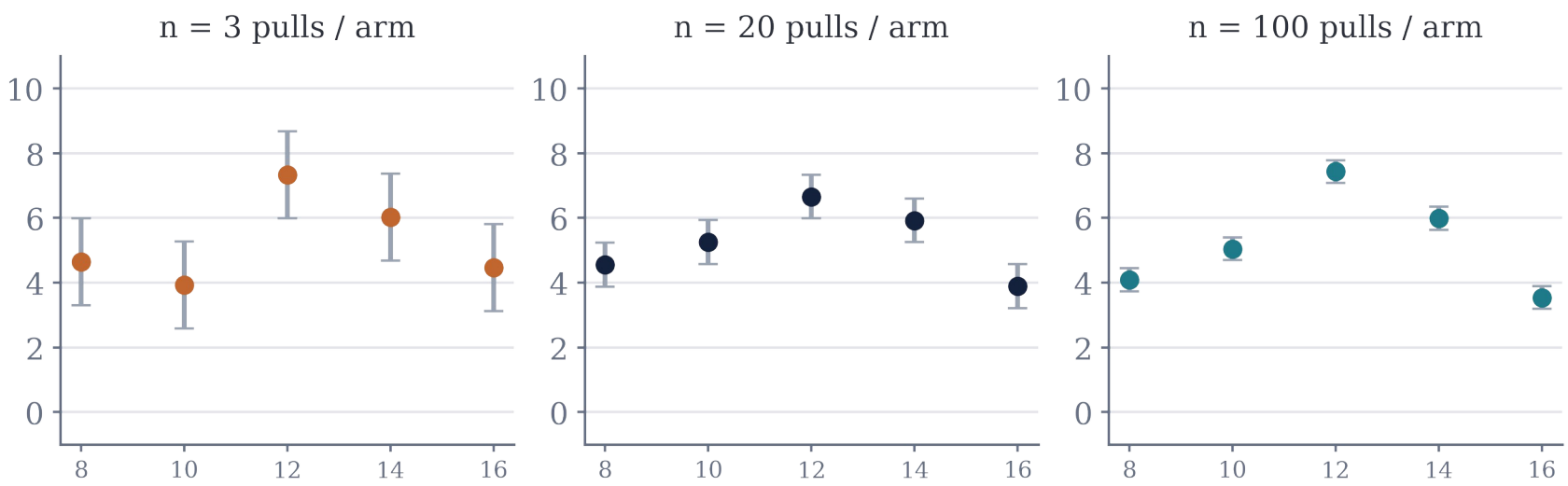
- "Optimism in the face of uncertainty": act as if every arm is as good as it plausibly could be.
- Rarely-pulled arms get a **large bonus**, pulling them toward exploration automatically.
- As N_i(t) grows, the bonus shrinks — exploration fades exactly where it is no longer needed.

Worked example: choosing the next pull



Price 12 has been pulled only 9 times and has a wide confidence bonus — its UCB value edges out price 14, which is pulled more but has less remaining uncertainty.

How the bound tightens with data



Exploration is directed and self-terminating: uncertainty itself is the reason to explore, and it disappears exactly where it is resolved.

Assessment

+ Strengths

- No randomness needed — fully deterministic given the history
- Achieves near-optimal (logarithmic) regret
- Exploration automatically targets the most uncertain arms

! Weaknesses

- Confidence bonus formula assumes a known noise model
- Can be conservative (over-explores) in practice
- Harder to extend to complex/structured action spaces than ε-greedy

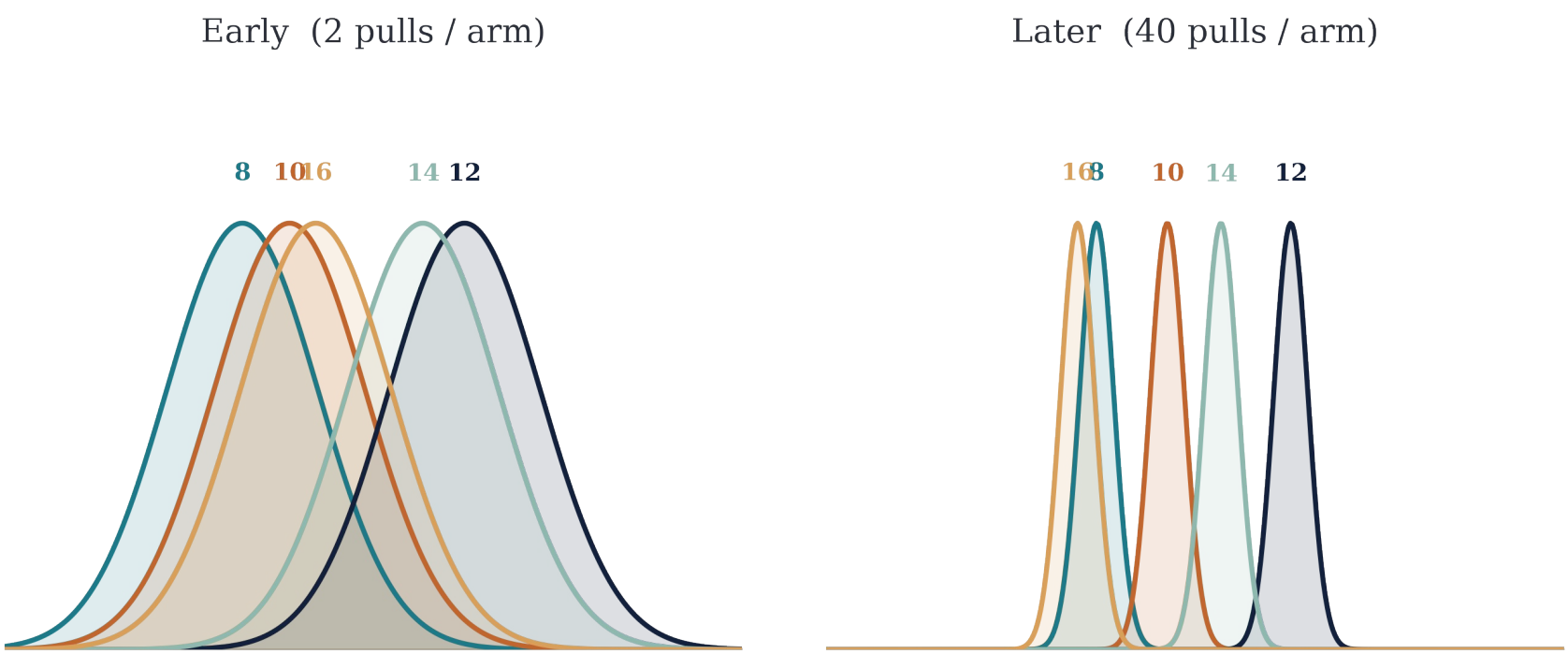
Explore by sampling belief itself

$$\theta_i \sim \text{Posterior}_i \qquad a_t = \arg\max_i \theta_i$$

Sample a plausible mean for every arm, then act greedily on the sample

- Maintain a full **posterior distribution** over each arm's mean, not just a point estimate.
- Each round, draw one random sample from each arm's posterior, and pull the arm with the highest sample.
- An arm gets pulled often either because it is genuinely good, or because uncertainty about it is still wide — both are legitimate reasons.

Posteriors sharpen with evidence



Early on, wide overlapping posteriors mean **any** arm could plausibly be sampled as the winner. Later, posteriors concentrate near the true means, and sampling reliably favors the best arm.

Assessment

+ Strengths

- Excellent empirical performance, often better than UCB in practice
- Naturally incorporates prior knowledge via the prior distribution
- Randomization handles ties and near-ties gracefully

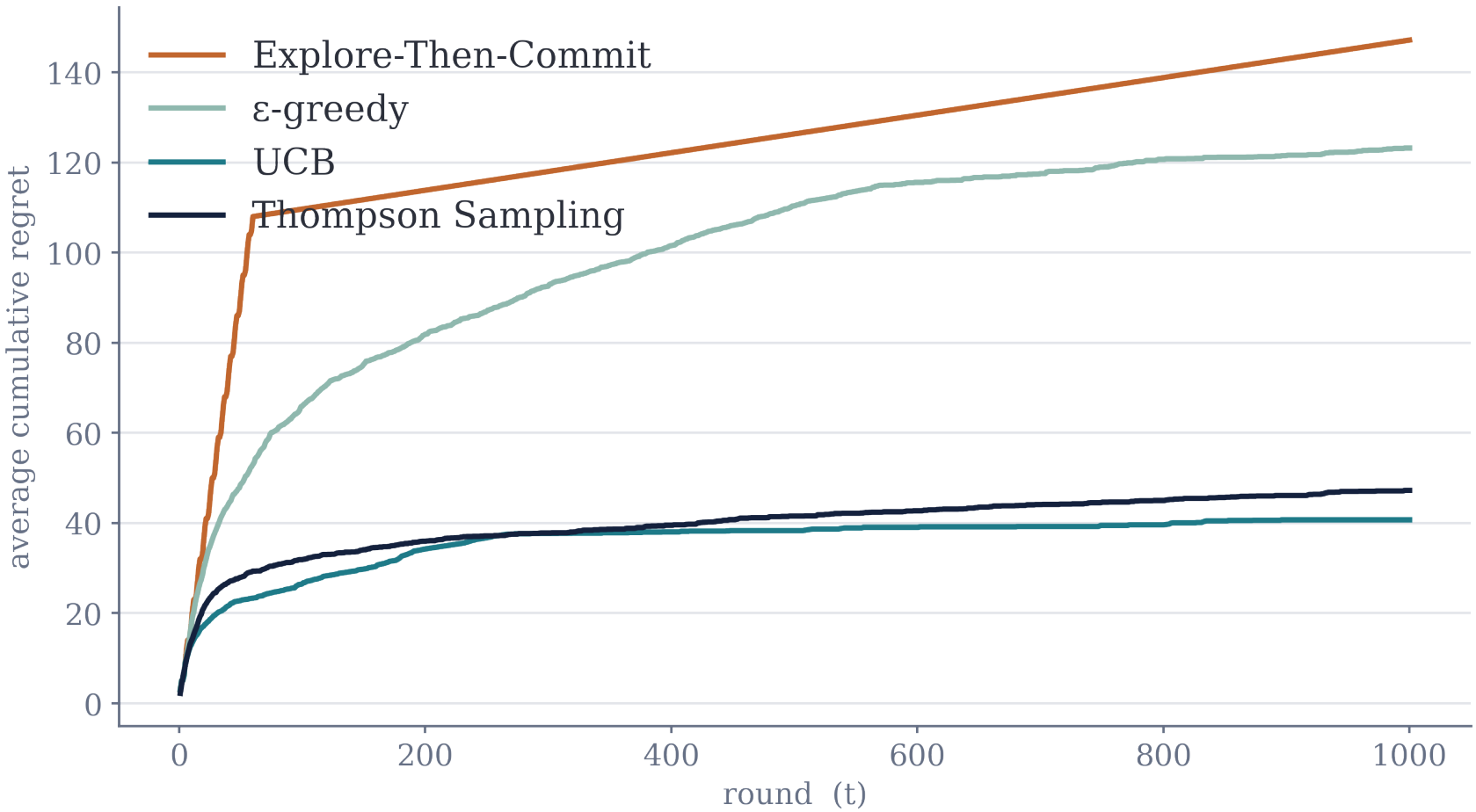
! Weaknesses

- Requires specifying a full probabilistic model and prior
- Posterior updates can be costly outside conjugate families
- Stochastic choices make behavior harder to audit than UCB

COMPARISON

All four algorithms, side by side

Average cumulative regret on the toy pricing problem, averaged over many independent runs.

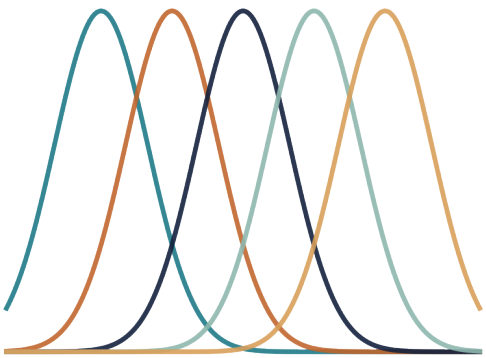


UCB and Thompson Sampling both achieve the characteristic **flattening (logarithmic) regret** of a well-designed bandit algorithm. ETC and ε-greedy both keep paying a steady, avoidable tax.

When the world fights back

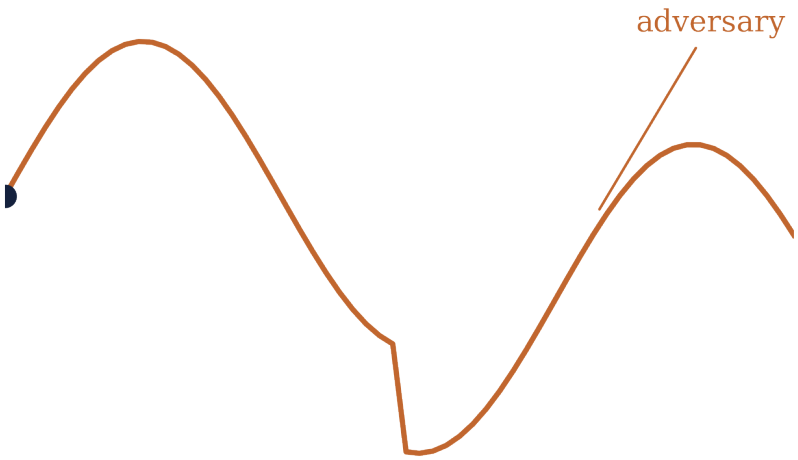
Every algorithm so far assumed each arm's reward comes from a fixed distribution. What if it doesn't?

Stochastic bandit



each arm: a fixed,
unchanging distribution

Adversarial bandit



best arm can change
round to round, even reactively

In the adversarial setting, rewards can be chosen by an adversary — possibly reacting to the learner's own past choices. UCB and Thompson Sampling's guarantees **no longer hold**.

EXP3

Exponential weights for an adversarial world

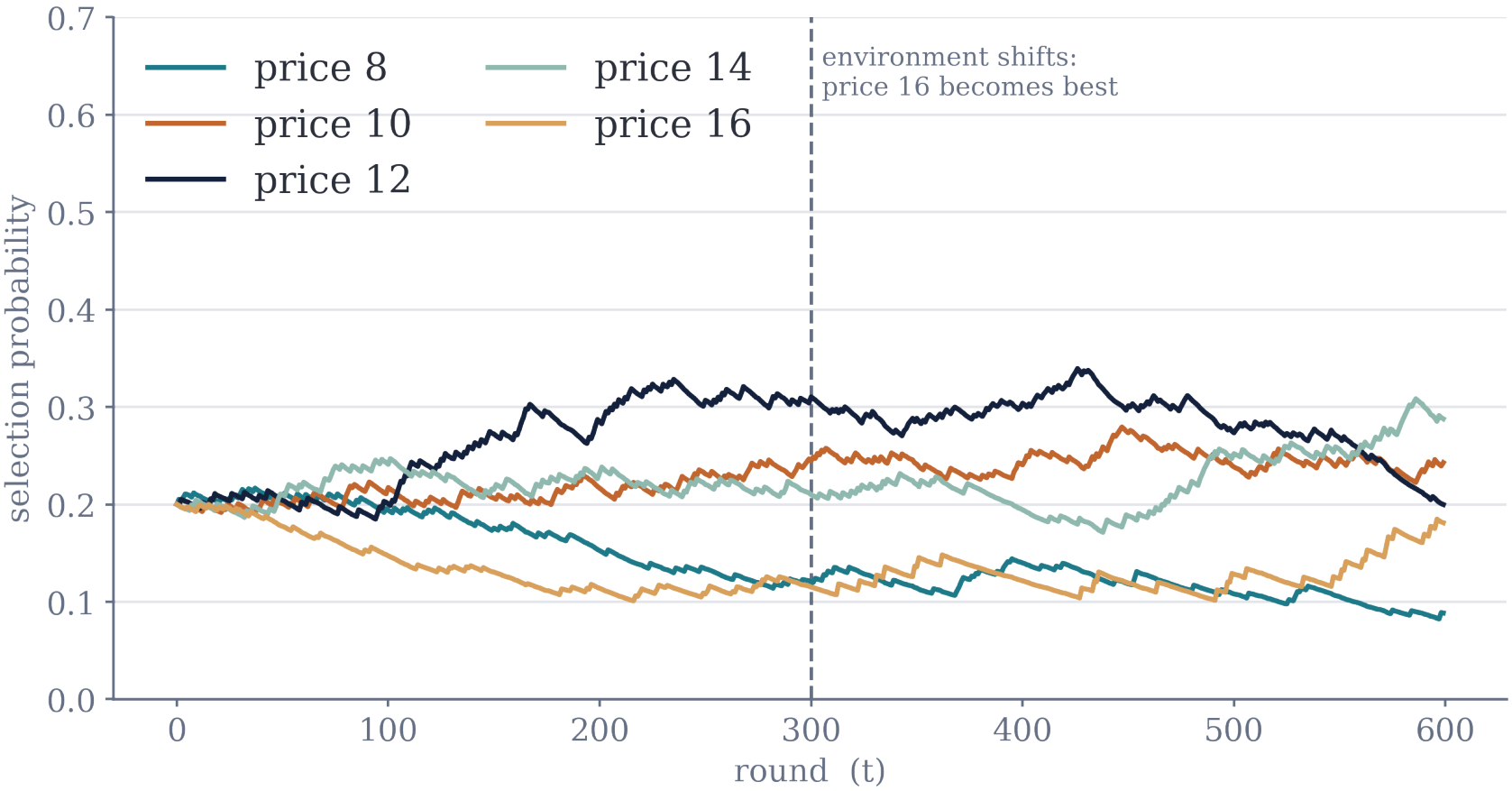
$$w_i(t+1) = w_i(t)\exp\left(\frac{\eta\hat{r}_i(t)}{K}\right)$$

Multiplicative weight update using an importance-weighted reward estimate

w_i(t) weight (unnormalized selection probability) of arm i
ŕ_i(t) importance-weighted reward estimate — only nonzero for the arm actually pulled
η learning rate controlling how fast weights adapt

- Every arm is pulled with **positive probability**, always — this is what makes EXP3 robust to an adversary.
- Good arms get exponentially more weight over time; bad arms decay away — but never to exactly zero.

Weights adapting to a changing world



When the best arm changes at round 300, EXP3's weights **shift accordingly** — the same multiplicative update that made it robust to adversaries also makes it adaptive to genuine non-stationarity.

Assumptions and computational notes

? Assumptions

- The decision is genuinely repeated, with feedback after each round
- Only the chosen arm's reward is observed (“bandit feedback”)
- Stochastic methods assume stationary reward distributions

$ Computational notes

- Per-round cost is tiny — usually O(K) to update and select
- No offline training phase; everything happens online
- Regret bounds, not runtime, are the typical performance currency

TAKEAWAY

What bandits teach us

“When learning and earning cannot be separated, exploration is not a phase you finish — it is a rate you tune, moment by moment, against exactly how uncertain you still are.”

Every algorithm here answers the same question differently: ETC fixes a budget in advance, ε-greedy randomizes, UCB is optimistic, Thompson Sampling is Bayesian, and EXP3 refuses to ever fully trust its own estimates.

7

PART 7 OF 8

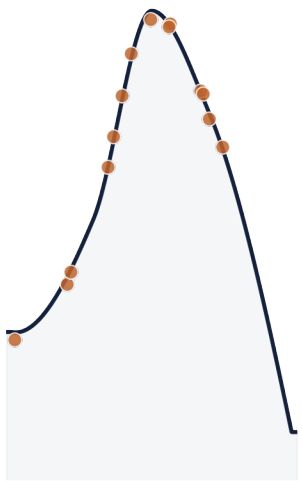
A Unifying View

Five methods, one question: how much to search, what information to trust, and how aggressively to commit. Here is the map that connects them.

Same problem, five perspectives

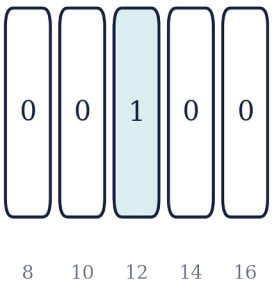
The same five candidate prices — through five different lenses

Population



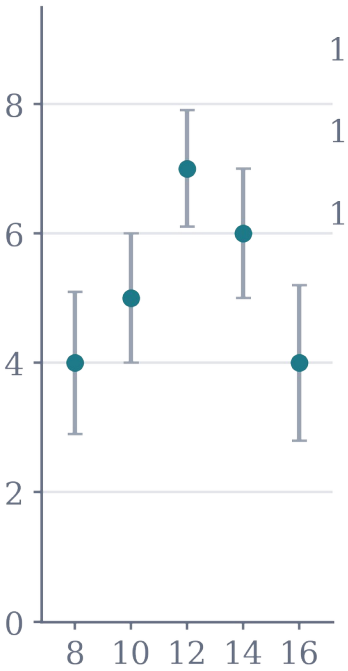
metaheuristics

Integer program



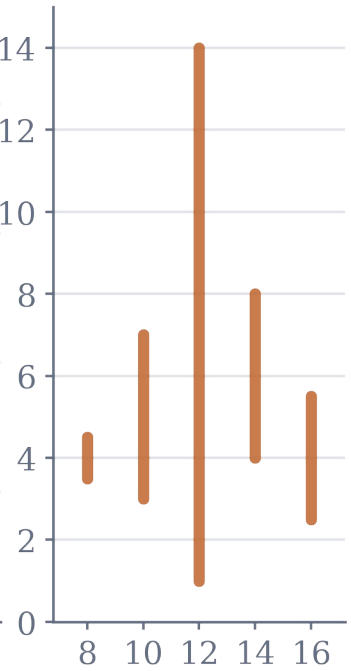
branch-and-bound

Expectation



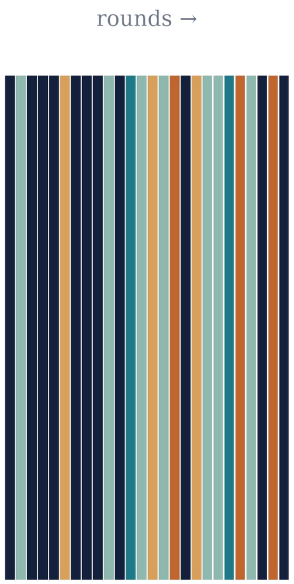
sample averages

Worst-case



uncertainty sets

Bandit



regret minimization

Nothing about the underlying decision changed across this entire lecture. What changed, each time, was **what the method assumed it knew, what it treated as random, and how it decided when to stop looking.**

What is explored, what is exploited

Method	What is explored	What is exploited
Metaheuristics	New candidates via mutation and crossover	Fitness, via selection pressure
Exact Optimization	Sub-regions of the search space via branching	A relaxation bound, via pruning
Stochastic Optimization	The demand distribution, via sampling	The sample-average estimate
Robust Optimization	The uncertainty set itself	A worst-case guarantee
Multi-Armed Bandits	Arms, via randomness or optimism	The current best empirical arm

Where the uncertainty comes from, and how exploration behaves

Method	Source of uncertainty	Character of exploration
Metaheuristics	Which region hides the optimum	Random / undirected
Exact Optimization	Which sub-region contains the optimum	Systematic
Stochastic Optimization	The true demand distribution	Data-driven
Robust Optimization	Which scenario will occur	Adversarially robust
Bandits (UCB / Thompson)	Each arm's true mean reward	Optimistic / probabilistic

Exploration is never just "trying random things" — each method has a distinct, principled reason for what it decides to try next.

What exploitation is willing to trust

Method	Exploitation is based on...	When exploration decreases
Metaheuristics	Relative fitness	As selection pressure concentrates the population
Exact Optimization	A provable bound	As branches get pruned and the tree shrinks
Stochastic Optimization	An estimated expectation	As more scenarios are sampled (never, within one solve)
Robust Optimization	A worst-case guarantee	It doesn't — the uncertainty set is fixed up front
Multi-Armed Bandits	A learned reward estimate	As confidence bounds tighten / posteriors sharpen

What “getting stuck” means, method by method

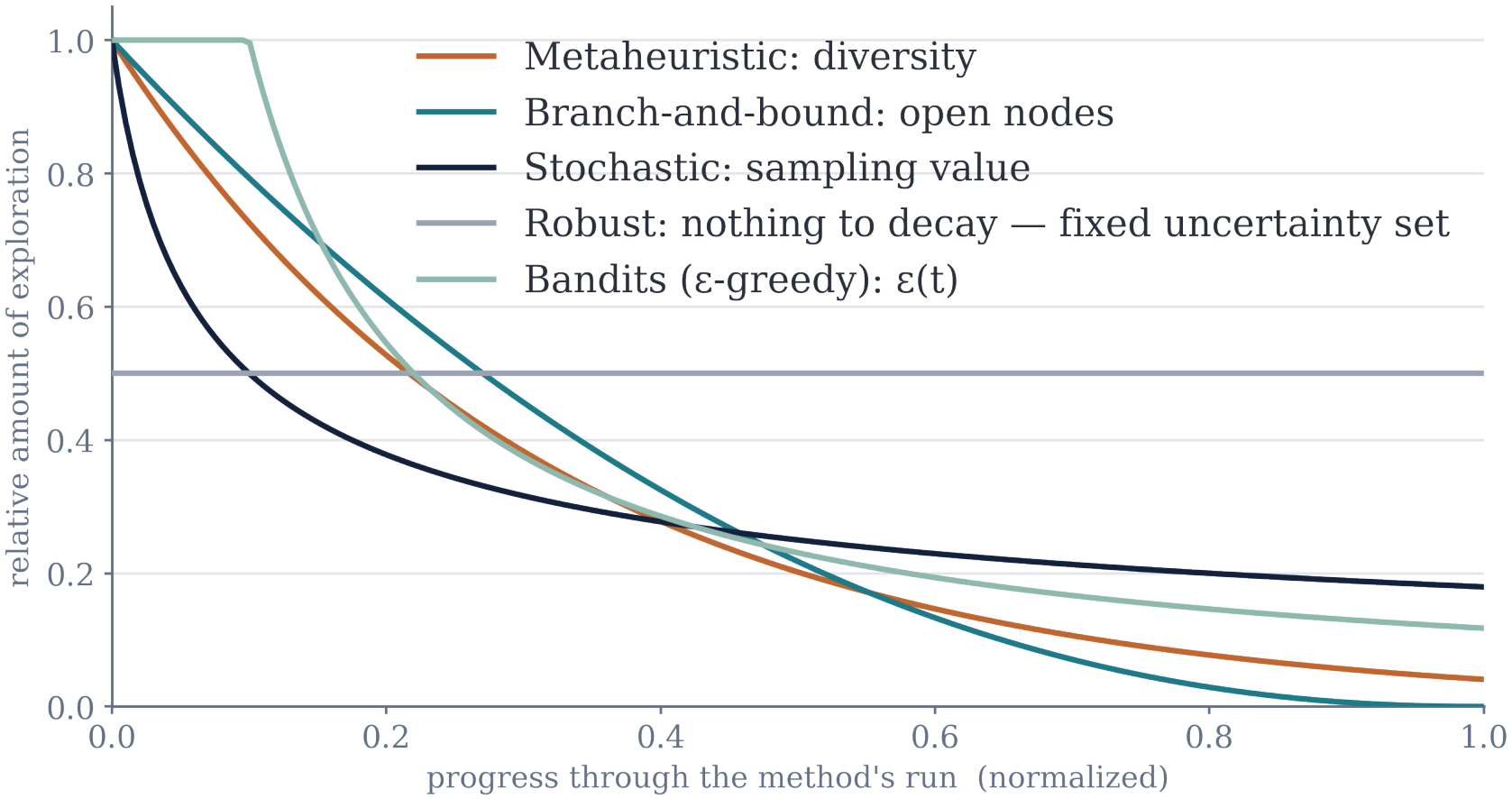
PREMATURE OR INSUFFICIENT EXPLORATION

Every method has its own failure signature for the same underlying mistake: trusting information before it is trustworthy.

- **Metaheuristics:** premature convergence — the whole population collapses onto one mediocre point.
- **Exact optimization:** no analogous failure — a loose bound just costs time, never correctness.
- **Stochastic optimization:** too few scenarios — a noisy estimate is trusted as if it were exact.
- **Robust optimization:** an uncertainty set that is too narrow — false confidence, not stagnation.
- **Bandits:** an arm is starved of pulls before its estimate can be trusted — the empirical analogue of “never getting a second chance.”

How exploration fades, method by method

Every method winds exploration down as it gains information — but on a completely different schedule.



Robust optimization is the outlier: it never decays, because it never learns — its uncertainty set is fixed before the method ever runs.

What counts as “information”

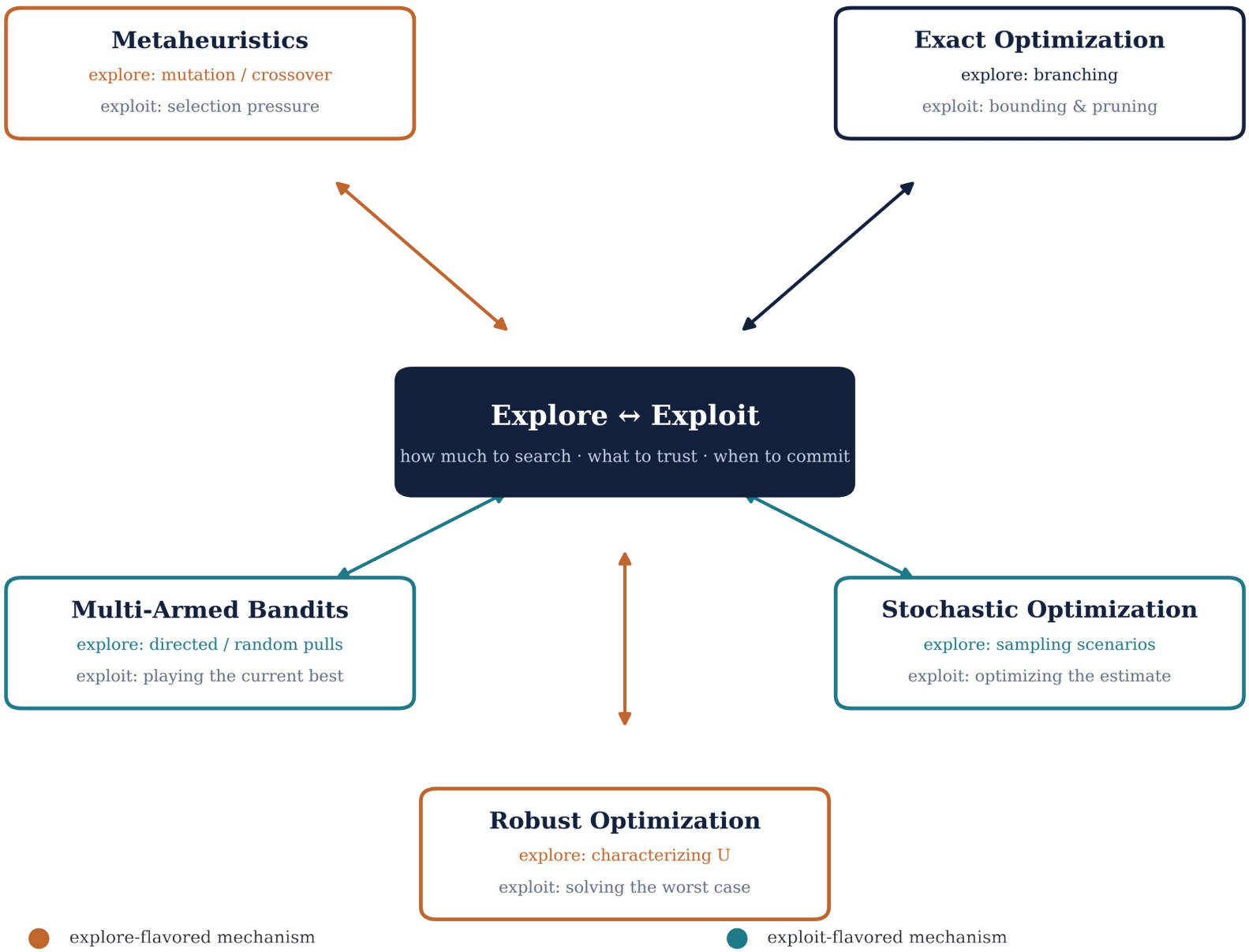
THE SAME WORD, FIVE DIFFERENT MEANINGS

Each method has a different currency for what it means to “learn something” during the search.

- For a **metaheuristic**, information is a fitness value attached to a candidate.
- For **exact optimization**, information is a tightened bound or a proven infeasibility.
- For **stochastic optimization**, information is a demand scenario.
- For **robust optimization**, information is knowledge about the shape of the uncertainty set itself — gathered before, not during, the optimization.
- For a **bandit**, information is a single observed reward, tied to exactly one action.

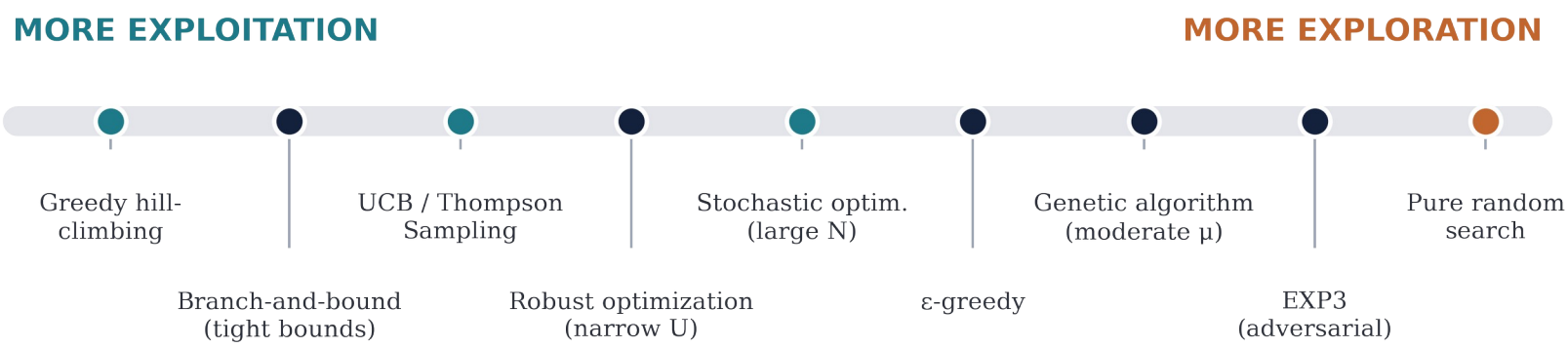
The cross-method conceptual map

One dilemma, five mechanisms



One spectrum, every method

Where each method — or a particular setting of it — sits between pure exploitation and pure exploration.



Notice that most methods are not fixed points on this spectrum — they are **positions a practitioner chooses**, via a mutation rate, a bound tightness, a sample size, an uncertainty budget, or an ε schedule.

Mechanisms and guarantees, side by side

Method	Explore mechanism	Exploit mechanism	Guarantee offered
Metaheuristics	Mutation, crossover	Selection pressure	None (heuristic)
Exact Optimization	Branching	Bounding & pruning	Proven optimality
Stochastic Optimization	Scenario sampling	Optimize the estimate	Statistical (as $N \to \infty$)
Robust Optimization	Model the uncertainty set	Solve the worst case	Worst-case guarantee
Multi-Armed Bandits	Optimism / randomness	Best empirical arm	Sublinear regret bound

Failure modes and information, side by side

Method	“Getting stuck” looks like...	Unit of information
Metaheuristics	Premature convergence	A fitness value
Exact Optimization	(slow, but never wrong)	A tightened bound
Stochastic Optimization	Trusting too small a sample	A demand scenario
Robust Optimization	An uncertainty set that's too narrow	Structure of the uncertainty set
Multi-Armed Bandits	An under-explored arm	One observed reward

The unifying idea

“Every method in this lecture is a different theory of when it is safe to stop learning and start acting — and every parameter inside it is a dial for that same one decision.”

Metaheuristics, exact search, stochastic and robust optimization, and bandits are not five unrelated tools. They are five different commitments about what to trust, and how much evidence is enough.

PART 8 OF 8

Practical Guidance & Synthesis

How to choose a method in practice, how methods combine, and the deeper ideas that connect all of optimization and learning under uncertainty.

EXPLORATION · EXPLOITATION

Questions to ask before choosing a method

1 About the decision

- Is it made once, or repeated with feedback?
- Is the space discrete, continuous, or combinatorial?
- How costly is a single evaluation?

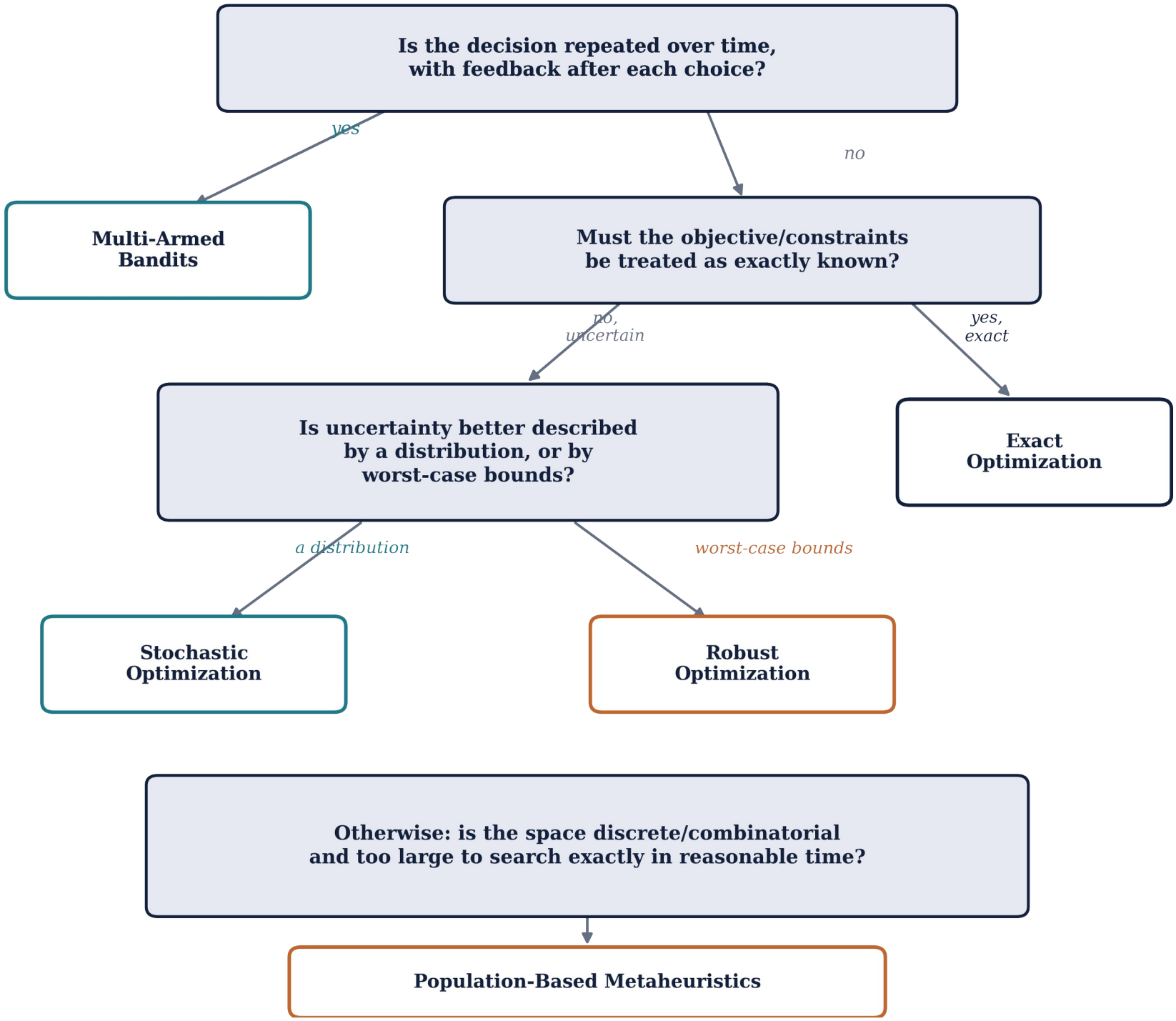
2 About the uncertainty

- Is it known, random, or adversarial?
- Can it be sampled or simulated?
- Is the downside risk tolerable or severe?

3 About the goal

- Is a proof of optimality required?
- Is “good enough, fast” acceptable?
- Is the answer needed once, or refined forever?

A decision guide



Method selection at a glance

Method	Repeated?	Optimality proof?	Handles adversarial?	Typical cost
Metaheuristics	No	No	No	Low–medium
Exact Optimization	No	Yes	No	Worst-case: high
Stochastic Optimization	No	Statistical	No	Medium
Robust Optimization	No	Worst-case	Yes	Low–medium
Multi-Armed Bandits	Yes	No (regret)	EXP3 only	Low per round

Methods are usually combined, not chosen

HYBRID STRATEGIES IN PRACTICE

In real systems, these methods are rarely used in isolation — each is often used to make another one faster or safer.

- A **metaheuristic** can supply exact optimization with a strong starting incumbent, pruning the search tree faster.
- **Stochastic optimization** can generate the scenario set that a **robust** method then protects against.
- A **bandit** algorithm can sit on top of any of the others, choosing *which* method or parameter setting to use online.
- Production pricing systems often run a **robust or exact layer for guardrails**, wrapped around a **bandit for continuous online refinement.**

Common pitfalls in practice

Under-exploring

- Tuning parameters on a single lucky run
- Trusting a small sample's ranking as final
- Stopping a search the moment it finds something merely good

Over-engineering

- Reaching for a bandit when the decision is one-shot
- Building a robust model no one can explain or trust
- Demanding exact optimality where an estimate would do

When simple beats sophisticated

A GROUNDING NOTE

Every method in this lecture has a regime where a much simpler method — or even a fixed rule — does just as well.

- If the gaps between options are large, **almost any** reasonable method finds the winner quickly.
- If the decision genuinely will not repeat, the elaborate machinery of a bandit is wasted effort.
- Sophistication earns its cost only when the underlying problem's uncertainty, scale, or stakes actually demand it.

DEEPER INSIGHT

Exploration is not free, exploitation is not safe

A NUANCE WORTH HOLDING ONTO

It is tempting to think of exploration as "costly but honest" and exploitation as "cheap but risky." Both halves of that intuition are incomplete.

- Exploration has an **opportunity cost** measured in the same currency as the final payoff — in a bandit, every exploratory pull is regret that can never be recovered.
- Exploitation is not safe either: acting confidently on a biased or under-sampled estimate is how every method in this lecture fails.
- The right question is never "explore or exploit?" — it is **"how much have I actually earned the right to trust what I currently believe?"**

The value of information vs. its cost

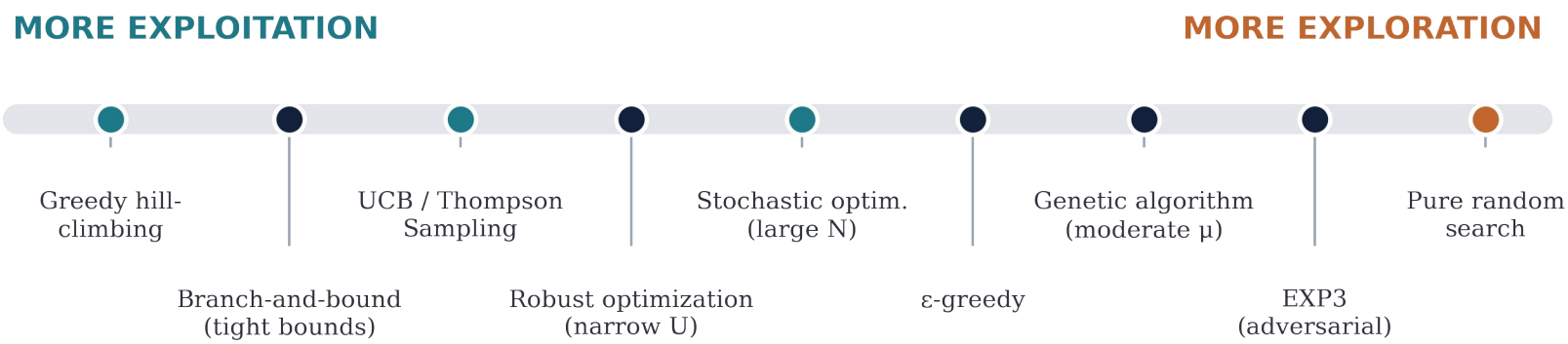
A UNIFYING ECONOMIC IDEA

Every method in this lecture is, underneath its specific mechanics, weighing the expected value of new information against the cost of acquiring it.

- A mutation's value is the chance it reveals a better region; its cost is one wasted evaluation if it doesn't.
- A demand scenario's value is a sharper estimate; its cost is the resource spent simulating or observing it.
- A bandit's exploratory pull's value is resolved uncertainty; its cost is measured, exactly, as regret.

Seen this way, exploration is never irrational — it is a **purchase**, and every method in this lecture is a different theory of what that purchase is worth.

One dilemma, five mechanisms, one question



From a greedy hill-climber to pure random search, every method in this lecture occupies — or lets its user choose — a position on exactly this one line.

The central question, answered five ways

“One central problem, five methodological perspectives, and one unifying question: how should an intelligent decision-maker balance learning what it does not know with using what it already believes?”

Population search explores through variation and exploits through selection. Exact search explores by branching and exploits by bounding. Stochastic optimization explores by sampling and exploits by estimating. Robust optimization explores the space of what could go wrong and exploits a guarantee. Bandits do both, every single round, forever.